Université de Montréal

La synthèse automatique de paraphrases comme outil de vérification des dictionnaires et grammaires de type Sens-Texte

par
François Lareau

Département de linguistique et de traduction
Faculté des arts et des sciences

Mémoire présenté à la Faculté des études supérieures
en vue de l'obtention du grade de M.A.
en linguistique

Septembre 2002

© François Lareau, 2002

Identification du jury

Université de Montréal
Faculté des études supérieures

Ce mémoire intitulé :
La synthèse automatique de paraphrases
comme outil de vérification des dictionnaires et grammaires
de type Sens-Texte

présenté par :
François Lareau

a été évalué par un jury composé des personnes suivantes :

.....
président-rapporteur

.....
directeur de recherche

.....
codirecteur

.....
membre du jury

Mémoire accepté le .....

Résumé

Ce mémoire présente un prototype de synthétiseur automatique de textes, *Sentence Garden*, destiné à servir de « laboratoire d'essais » pour la vérification de modèles linguistiques Sens-Texte. L'originalité de ce synthétiseur réside principalement dans le fait qu'il vise à produire non pas une seule phrase, mais toutes les phrases possibles qui expriment un même message. Les modèles Sens-Texte qu'il utilise sont présentés d'abord d'un point de vue théorique, puis du point de vue de leur encodage dans *Sentence Garden*.

Mots-clés : génération de texte, linguistique informatique, théorie Sens-Texte, lexicologie formelle, sémantique computationnelle, syntaxe de dépendance, linguistique théorique

Abstract

This dissertation presents an automatic text synthesis prototype system, *Sentence Garden*, which is intended to be used as a "test lab" for the validation of Meaning-Text linguistic models. The originality of this system lies mainly in the fact that it aims at producing not only one sentence, but all possible sentences that express a given meaning. Meaning-Text models are introduced first from a theoretical point of view, then from the viewpoint of their encoding in *Sentence Garden*.

Keywords: text generation, computational linguistics, Meaning-Text Theory, formal lexicology, computational linguistics, dependency syntax, theoretical linguistics

Table des matières

Identification du jury . . . . . **ii**

Résumé . . . . . **iii**

Abstract . . . . . **iv**

Table des matières . . . . . **v**

Liste des figures et des règles linguistiques . . . . . **vii**

Liste des abréviations et conventions d'écriture . . . . . **viii**

Remerciements . . . . . **ix**

Chapitre I. Introduction **11**

1 - But et intérêt de la recherche 12

2 - Cadre de travail : la théorie Sens-Texte 13

3 - Générateur de textes *vs* synthétiseur de textes 14

4 - La synthèse de paraphrases 17

Chapitre II. Présentation de *Sentence Garden* **23**

1 - Utilisation du système 23

2 - Les modules linguistiques 25

3 - Les RSém sources et les résultats de la synthèse 26

4 - Un exemple complet de synthèse 27

Chapitre III. Les modèles Sens-Texte **32**

1 - Les représentations linguistiques 33

1.1 - Caractérisation formelle des représentations linguistiques en TST 33

1.2 - L'encodage des représentations linguistiques dans *Sentence Garden* . . . 36

2 - La grammaire 39

2.1 - Caractérisation formelle des grammaires Sens-Texte 39

2.2 - L'encodage de la grammaire dans *Sentence Garden* 41

3 - Le dictionnaire 44
3.1 - Caractérisation formelle du *DiCo* 44
3.2 - L'encodage du dictionnaire dans *Sentence Garden* 49
3.3 - Les patrons de régime 53
3.4 - L'importation et la validation du *DiCo* 57
3.5 - L'encodage du dictionnaire sous forme de règles 60
Chapitre IV. La synthèse dans *Sentence Garden* 63
1 - L'algorithme général de synthèse dans *Sentence Garden* 63
2 - Le parcours de la représentation source 64
2.1 - Problématique 64
2.2 - Le choix du nœud d'entrée 67
2.3 - L'exploration de la représentation source 69
2.4 - La notion de parcours dans un synthétiseur de paraphrases 70
3 - La procédure de transition 71
4 - La procédure de matching des règles 72
4.1 - Le matching des parties gauche et droite des règles 72
4.2 - La vérification des conditions d'application des règles 73
4.3 - Les problèmes liés à la procédure de matching des règles 74
4.4 - Vers un matching souple des règles linguistiques 76
5 - La procédure d'application des règles 77
Chapitre V. Conclusion. 79
Index des notions 81
Références 83
Annexe 1. Un article du *DiCo* xi
Annexe 2. Le logiciel *JEditGraph* xii
Annexe 3. Quelques règles de grammaire xiii
Annexe 4. Une récolte xiv
Annexe 5. L'importation du dictionnaire xxv
Annexe 6. Un rapport de *Customs* xxvi

Liste des figures et des règles linguistiques

Liste des figures

Figure 1. Les principales composantes d'un générateur de textes. 16
Figure 2. La synthèse selon la TST 17
Figure 3. La synthèse vue comme un ensemble de microsynthèses 18
Figure 4. La microsynthèse vue comme un ensemble de transitions 19
Figure 5. La transition vue comme un ensemble de transitions élémentaires 20
Figure 6. La microsynthèse vue comme un ensemble de nanosynthèses 21
Figure 7. Utilisation de *Sentence Garden* 25
Figure 8. La RSém de départ 28
Figure 9. Un nœud (syntaxique profond) 33
Figure 10. Un arc (syntaxique de surface). 33
Figure 11. Une représentation sémantique 35
Figure 12. Une représentation syntaxique profonde (partielle). 36
Figure 13. Un exemple de règle de grammaire 41
Figure 14. Une lacune dans les patrons de régime 54
Figure 15. Deux types de parcours des arbres syntaxiques 65
Figure 16. Deux nœuds d'entrée potentiels 68
Figure 17. Deux familles de paraphrases. 68
Figure 18. Un article du *DiCo* dans *FileMaker Pro* : REGRET1 xi
Figure 19. L'environnement *JEditGraph* xii

Liste des règles linguistiques

Règle 1. Le sens de GLOUTON. 62
Règle 2. Une règle SyntS du japonais 76
Règle 3. RelSém 1 ⇔ RelSyntP I xiii
Règle 4. RelSém 2 ⇔ RelSyntP II xiii
Règle 5. RelSém 1 ⇔ RelSyntP ATTR. xiii
Règle 6. RelSém 2 ⇔ RelSyntP ATTR. xiii
Règle 7. Introduction d'un verbe support xiii

Liste des abréviations et conventions d'écriture

Abréviations

ActSém . . . . . *actant sémantique*
ActSyntP . . . . . *actant syntaxique profond*
ActSyntS . . . . . *actant syntaxique de surface*
BD . . . . . *base de données*
DEC . . . . . *Dictionnaire explicatif et combinatoire*
DiCo. . . . . *Dictionnaire des dérivations et collocations*
FL. . . . . *fonction lexicale*
LAF . . . . . *Lexique actif du français*
MST . . . . . *modèle Sens-Texte*
RelSém. . . . . *relation sémantique*
RelSyntP . . . . . *relation syntaxique profonde*
RelSyntS . . . . . *relation syntaxique de surface*
RMorphP . . . . . *représentation morphologique profonde*
RSém . . . . . *représentation sémantique*
RSyntP . . . . . *représentation syntaxique profonde*
RSyntS . . . . . *représentation syntaxique de surface*
SSém . . . . . *structure sémantique*
SSém-Comm . . . . . *structure sémantico-communicative*
SSém-Rhét . . . . . *structure sémantico-rhétorique*
SSynt-AnaphP . . . . . *structure syntaxico-anaphorique profonde*
SSynt-CommP . . . . . *structure syntaxico-communicative profonde*
SSyntP . . . . . *structure syntaxique profonde*
TST . . . . . *théorie Sens-Texte*

Conventions d'écriture

X . . . . . *variable*
‹tigre› . . . . . *sémantème*
REGRET1 . . . . . *lexie avec son numéro d'identification*
ABATTEMENT^{2}. . . . . *lexie avec son numéro d'identification de vocable*
⌜À PROPOS DE⌝ . . . . . *phrasème*
microsynthèse . . . . . *nouveau terme technique*
graphe . . . . . *classe d'objets*
`microsynthèse(X)` . . . . . *code source, pseudo-code, extraits de fichiers*
semr5 . . . . . *lien hypertexte*

Chapitre I. Introduction

Le travail qui fait l'objet de ce mémoire relève du domaine de la linguistique informatique, plus précisément de la génération automatique de textes (Reiter & Dale, 2000; Polguère, 1997b). Nous présentons dans ce mémoire un système informatique appelé *Sentence Garden*[1]. Il s'agit d'un prototype de synthétiseur automatique de textes destiné à être utilisé comme outil de vérification des modèles linguistiques Sens-Texte. Il se distingue de la majorité des autres systèmes de type Sens-Texte développés auparavant en ce qu'il vise à produire toutes les paraphrases exprimant un message donné plutôt qu'une seule phrase (toutefois, dans sa version actuelle, le prototype n'effectue pas la synthèse au-delà du niveau syntaxique profond).

Puisque notre travail relève de la linguistique informatique, ce mémoire s'adresse au moins autant à des linguistes qu'à des informaticiens. Pour cette raison, nous avons tenté d'éviter le plus possible le jargon informatique et les discussions trop techniques par rapport à l'implémentation de notre système. Tous les programmes constituant *Sentence Garden* peuvent être téléchargés de la Toile à partir de la page http://www.fas.umontreal.ca/ling/olst/lareau/fra/garden/. Les informaticiens pourront ainsi consulter directement le code source des programmes.

Néanmoins, il est impossible de parler de linguistique de façon claire et cohérente sans utiliser une terminologie solide et explicite. La terminologie utilisée tout au long de cet ouvrage est principalement empruntée à la théorie Sens-Texte (*cf.* §2, p.13). Cependant, de nouveaux termes ont dû être introduits afin de

1. *Jardin de phrases*, en français. Nous avons choisi d'écrire tous les programmes, de les commenter et de les nommer en anglais, de façon à en assurer la plus grande diffusion possible.

combler des vides dans le système conceptuel de cette théorie et de la linguistique en général. Nous avons essayé de mettre en place une terminologie aussi propre que possible, exempte d'ambiguïté comme de synonymie, et qui respecte l'usage courant en linguistique. Tout écart par rapport à ces objectifs est signalé par une note qui en avertit le lecteur. Par ailleurs, la première fois qu'un nouveau terme technique apparaît dans le texte, il est mis en évidence par une **police spéciale**.

L'organisation de ce mémoire est la suivante. Les prochaines sections du présent chapitre situeront notre travail de façon générale et fourniront les bases théoriques nécessaires à notre exposé. Le chapitre II présentera dans ses grandes lignes le système informatique *Sentence Garden*. Dans le chapitre III, nous verrons de quoi sont composés les modèles linguistiques que nous utilisons et comment ils sont encodés dans notre système. C'est dans le chapitre IV que nous verrons comment ces modèles linguistiques sont appliqués par *Sentence Garden*. Enfin, nous reviendrons au chapitre V sur les points importants de ce mémoire.

1 But et intérêt de la recherche

Bien que le système informatique présenté soit un synthétiseur de textes, le but que nous poursuivons n'est pas de faire de la génération de textes en soi. Nos buts sont plutôt les suivants :

1) Démontrer la validité de la théorie Sens-Texte (*cf.* §2, ci-dessous) et son utilité pour le traitement automatique des langues.

2) Offrir aux linguistes des outils informatiques permettant de construire et surtout vérifier leurs modèles linguistiques.

3) Développer un synthétiseur qui simule le comportement de la langue, et non celui d'un locuteur, c'est-à-dire un système qui ne produise non pas une seule phrase, mais toutes les paraphrases exprimant un sens donné.

4) Explorer de nouvelles méthodes d'encodage des dictionnaires.

L'originalité de *Sentence Garden* se situe principalement dans le fait qu'il vise à produire un ensemble de paraphrases. Peu de générateurs possèdent cette caractéristique, puisque la plupart des générateurs existants simulent le comportement d'un locuteur; ils ne produisent toujours qu'une seule expression, sans considérer les autres alternatives. Nous croyons que la production en parallèle de plusieurs phrases exprimant le même sens est le meilleur moyen d'atteindre les objectifs que nous nous sommes fixés. D'abord, la paraphrase est au cœur de la théorie Sens-Texte. Ensuite, les modèles linguistiques développés par d'éventuels utilisateurs de notre système sont des modèles fonctionnels des langues et non pas des locuteurs. Or, les langues présentent justement un pouvoir paraphrastique considérable. Il faut que cette capacité à produire plusieurs phrases pour exprimer le même sens se retrouve dans un système visant à valider des modèles linguistiques.

Enfin, ce synthétiseur a été réalisé dans le cadre d'un projet de recherche des professeurs Polguère et Kittredge de l'Université de Montréal subventionné par le CRSNG du Canada[1].

2 Cadre de travail: la théorie Sens-Texte

Nous tiendrons pour acquis que le lecteur connaît déjà la théorie Sens-Texte [TST]. De nombreux ouvrages la présentent de façon plus ou moins détaillée et nous croyons qu'il serait inutile d'en offrir une énième introduction. Nous nous contenterons donc de renvoyer tout lecteur étranger à cette théorie à (Mel'čuk, 1997), ou encore (Polguère, 1998) pour une introduction plus compacte.

Les quatre principales raisons pour lesquelles nous avons choisi la TST comme cadre théorique sont les suivantes:

1) La paraphrase est au cœur de la TST. Les théories linguistiques actuelles ayant cette caractéristique sont rares, et c'était une caractéristique nécessaire, compte tenu de notre sujet de recherche.

1. Subvention no. OGPO200713.

2) C'est une théorie formelle. Sans formalisation, la linguistique informatique est impossible.

3) Elle permet un traitement uniforme de phénomènes linguistiques de natures variées (sémantique, syntaxique, morphologique et phonologique) dans toutes les langues. Nous voulions justement que *Sentence Garden* puisse fonctionner pour n'importe quelle langue.

4) Elle offre, en plus d'un métalangage formel puissant, une terminologie bien développée.

Un certain nombre de systèmes informatiques basés sur la théorie Sens-Texte ont vu le jour ou sont en cours de réalisation: *GOSSIP* (Iordanskaja *et al.*, 1988), *FoG* (Goldberg *et al.*, 1994), *LFS* (Iordanskaja *et al.*, 1992), *RealPro* (Lavoie & Rambow, 1997), *ETAP* (Apresjan *et al.*, 1992), *AlethGen* (Coch, 1996) et *MATE* (Bohnet *et al.*, 2000), pour n'en nommer que quelques-uns. *MATE* est sans doute celui qui a le plus en commun avec *Sentence Garden*, aussi nous en reparlerons à quelques reprises. De plus, (Boyer & Lapalme, 1985) décrit un système qui n'a pas été réalisé mais qui ressemble beaucoup à *Sentence Garden*. Parmi tous ces systèmes, certains sont des générateurs (*GOSSIP*, *FoG* et *LFS*), d'autres sont des synthétiseurs (*RealPro*, *AlethGen* et *MATE*). *ETAP*, quant à lui, est un système de traduction automatique. Nous allons maintenant expliquer quelle est la différence entre un synthétiseur et un générateur.

3 Générateur de textes *vs* synthétiseur de textes

Un **générateur de textes** est un système qui, sous certains stimuli du monde réel, conçoit un message à communiquer, puis produit le ou les textes qui lui permettent de l'exprimer. Un tel système utilise non seulement des données linguistiques, mais aussi extralinguistiques (connaissances sur le monde réel, modèles cognitifs, etc.). Un générateur doit effectuer deux tâches principales: ***1)*** calculer quoi dire et ***2)*** calculer comment le dire.

Nous appellerons **générateur de messages** le module d'un générateur de textes qui calcule quoi dire, c'est-à-dire qui réagit à certains stimuli en construisant un message linguistique (*cf.* Polguère, 1990), qu'il encode dans une représentation sémantique [RSém].

Un **synthétiseur de textes** est un système (ou le module d'un système) qui ne prend en charge que la réalisation d'un message. Il prend donc en entrée une RSém et donne en sortie un ou plusieurs textes qui expriment celle-ci. Nous reviendrons sur la notion de synthèse à la prochaine section.

Une distinction importante est à faire : il ne faut surtout pas confondre *synthèse de textes* et *synthèse de la parole*, qui désigne plutôt le passage d'une représentation phonologique (ou graphémique) à une suite de réalisations sonores. Un synthétiseur de la parole peut être intégré à un synthétiseur de textes. Notre utilisation du terme *synthétiseur*, emprunté à (Polguère, 1990), semble s'écarter un peu de l'usage courant puisqu'on utilise très souvent ce terme pour parler de *synthétiseurs de la parole*. Or, **Sentence Garden** ***ne fait pas la synthèse de la parole***.

Ces deux composantes, générateur de message et synthétiseur de textes, sont les deux pièces maîtresses d'un générateur de textes, mais ne suffisent pas à constituer un système complet. Il faudrait leur ajouter d'autres composantes, comme, par exemple, un module s'occupant de l'enchaînement des phrases, un autre qui choisisse parmi diverses variantes stylistiques, etc. Nous ne parlerons pas de ces modules. Notre but était avant tout de préciser ce que nous entendons par

synthétiseur de textes. La figure suivante illustre la différence entre *générateur de messages*, *générateur de textes* et *synthétiseur de textes*.

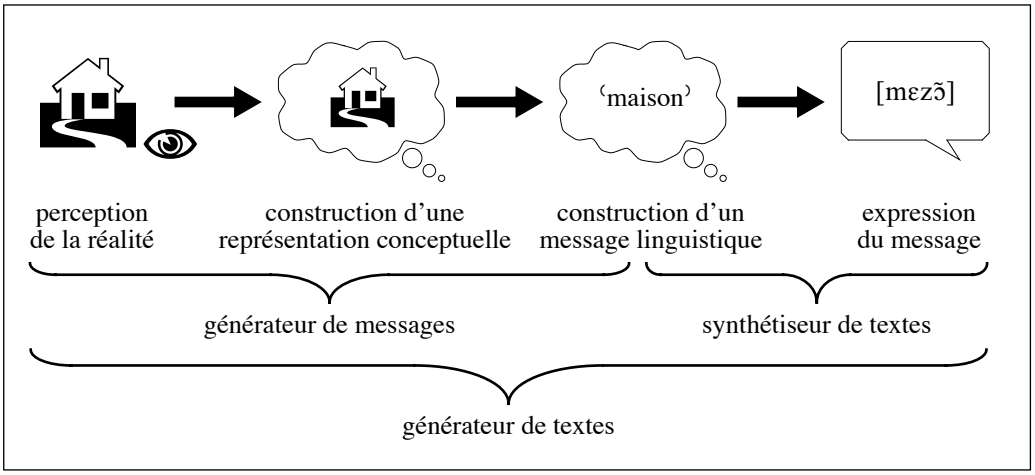


Figure 1. Les principales composantes d'un générateur de textes

Sentence Garden n'est pas un générateur, mais bien un synthétiseur de textes. Dans sa version complète, ce système prendrait en entrée une RSém déjà formée (soit par un humain, soit par un autre système informatique) et donnerait en sortie tous les textes qui expriment le message encodé dans cette RSém, en n'utilisant que des connaissances linguistiques (c'est-à-dire grammaticales et lexicales). Les textes produits seraient monophrastiques, puisque la TST n'offre pas encore de façon élégante de modéliser la production de textes pluriphrastiques. Toutefois, le prototype réalisé ne fait pas la synthèse au-delà du niveau syntaxique profond. La sortie n'est donc pas un ensemble de paraphrases, mais seulement un ensemble de représentations syntaxiques profondes [RSyntP]. Il est important de bien comprendre que *Sentence Garden* ne modélise aucunement un locuteur. Il n'est rien d'autre qu'un moteur qui fait tourner les règles d'un modèle Sens-Texte [MST]. Or, un MST est un modèle fonctionnel d'une ***langue***. C'est pour cette raison que *Sentence Garden* vise à produire non pas un seul texte, comme un locuteur le ferait, mais tous les textes que le MST dont il dispose lui permet de produire (ces textes étant des paraphrases, puisqu'ils sont des réalisations différentes d'une même RSém). En ce sens, dans une application pratique, il pourrait être utilisé comme une sorte de « serveur linguistique » fournissant toutes les réalisations possibles d'une RSém. Il reviendrait alors au générateur l'utilisant

de choisir parmi les différentes paraphrases proposées par *Sentence Garden*. Nous verrons à la section suivante en quoi consiste exactement la synthèse de paraphrases.

4 La synthèse de paraphrases

Dans le titre de ce mémoire, nous parlons de *synthèse de paraphrases*. Une remarque à ce sujet s'impose afin d'éviter un malentendu. En effet, il se fait actuellement beaucoup de travaux sur la reformulation : on développe des systèmes qui prennent en entrée une phrase et produisent d'autres phrases exprimant le même sens. Donc, dès qu'on dit *paraphrase*, tout le monde semble comprendre *reformulation*. Or, **Sentence Garden *n'est pas un système de reformulation.*** Nous aurions peut-être pu mettre ce fait en évidence en utilisant l'expression *synthèse parallèle de paraphrases* dans notre titre. Nous ne proposons toutefois pas de retenir cette expression tarabiscotée ; nous préférons parler simplement de synthèse, et nous insistons sur le fait que dans la langue (et non dans la parole, au sens saussurien des termes *langue* et *parole*), la production de paraphrases est inséparable de l'idée de synthèse.

La **synthèse**, dans le cadre de la TST, s'effectue de façon modulaire, en passant par une série de représentations intermédiaires.

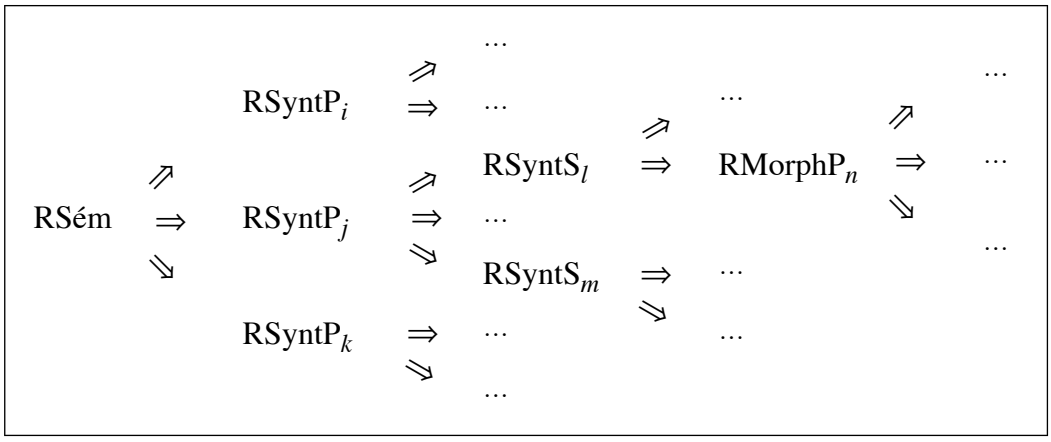


Figure 2. La synthèse selon la TST

Puisqu'elle s'effectue en passant par des représentations intermédiaires, la synthèse peut être considérée comme une série de **microsynthèses**, appliquées récursivement aux représentations construites par les microsynthèses précédentes. Une microsynthèse assure le passage entre deux niveaux adjacents seulement. La figure suivante isole une des microsynthèses qui composent la synthèse illustrée à la figure 2.

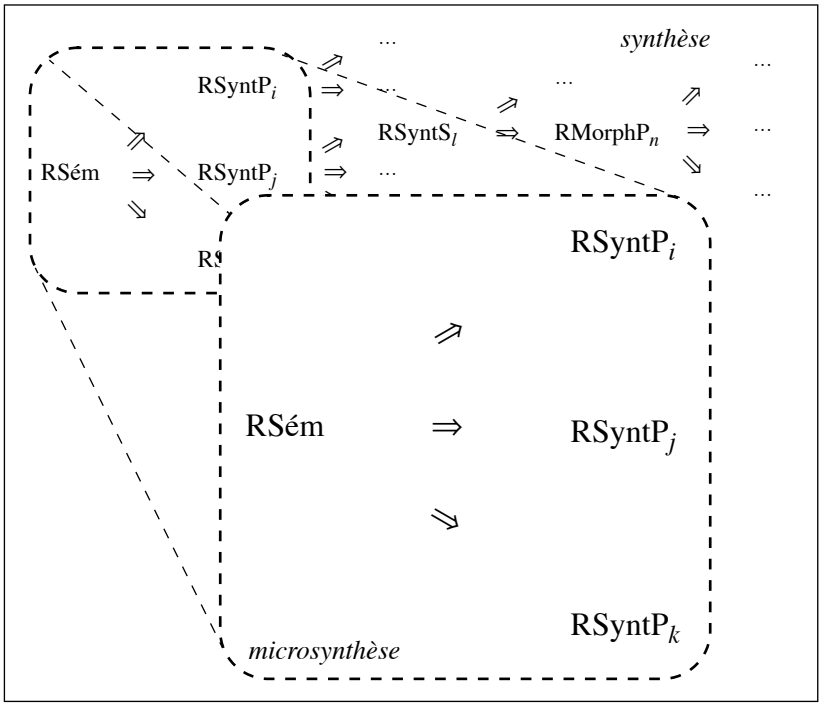


Figure 3. La synthèse vue comme un ensemble de microsynthèses

Alors que la synthèse peut être définie comme une série de microsynthèses, la microsynthèse, à son tour, repose sur une opération plus simple, la transition. La **transition** est l'opération qui consiste à construire, à partir d'une représentation de niveau *N* déjà existante que nous appellerons la (représentation) **source**, une représentation correspondante de niveau adjacent, que nous appellerons le **produit** (ou la représentation produit). Il existe deux types de transitions: la transition synthétique, orientée du sens vers le texte (c'est-à-dire que le produit est de niveau *N+1*) et la transition analytique, dont l'orientation est inverse (le produit

étant de niveau *N-1*). La figure suivante reprend la microsynthèse isolée à la figure 3 au sein de la synthèse et montre qu'elle est composée de trois transitions.

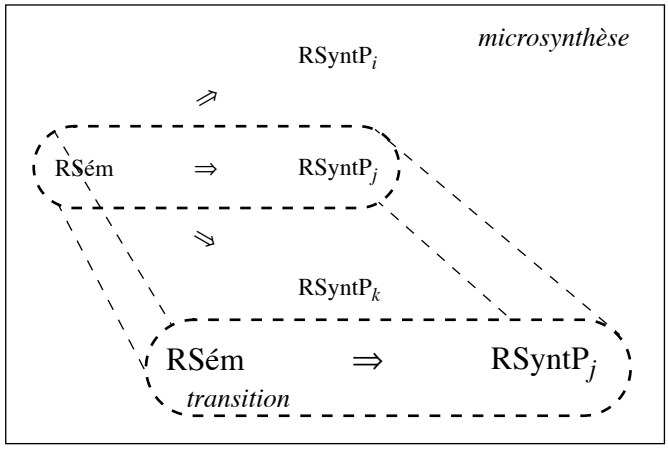


Figure 4. La microsynthèse vue comme un ensemble de transitions

Trois remarques terminologiques s'imposent au sujet de la transition :

1) Comme nous ne parlerons aucunement de transition analytique dans ce mémoire, nous nous permettrons d'utiliser simplement le terme *transition* au lieu de *transition synthétique*.

2) Nous nous permettrons un autre raccourci terminologique en utilisant les expressions *la source de R* et *le produit de R* (où *R* est une représentation). Ces expressions doivent être comprises respectivement comme *la représentation source de la transition dont R est la représentation produit* et *la représentation produit de la transition dont R est la représentation source*.

3) Le terme *transition* étant difficilement verbalisable, il nous force à utiliser des tournures assez lourdes dans notre exposé. Nous emploierons donc le verbe **réaliser**[1] de la façon suivante : ‹réaliser R_1 par R_2› = ‹effectuer la transition synthétique entre R_1 et R_2›.

Cela dit, les langues ayant un grand pouvoir paraphrastique, il est assez rare que pour une microsynthèse donnée il n'y ait qu'une seule transition possible[2]. Nous

1. On trouve parfois le terme *traduire* dans la littérature Sens-Texte pour exprimer ce sens. Cette pratique doit être évitée car elle porte à confusion. En effet, il ne s'agit aucunement de traduire d'une langue à une autre les représentations.

appellerons la situation où plusieurs transitions sont possibles à partir d'une même source la **concurrence**, et nous parlerons de **transitions concurrentes**. Alors que la majorité des synthétiseurs existants s'efforcent de ne choisir toujours qu'une seule des transitions concurrentes, de façon à n'obtenir qu'un seul texte, *Sentence Garden* applique indifféremment chaque transition, produisant ainsi tous les textes pouvant exprimer le message encodé dans la RSém de départ (selon les limites de la grammaire et du dictionnaire disponibles).

La transition entre une source et un produit est régie par un MST, c'est-à-dire un ensemble de règles linguistiques (*cf.* ch. III, p. 32). Un MST est un peu comme une recette qui indique comment construire un produit à partir d'une source donnée. Toutefois, le nombre de sources et de produits possibles étant infini, les MST ne peuvent pas prendre en charge des représentations linguistiques entières. Les règles, plutôt que de viser des représentations, visent les **éléments** de ces représentations : les nœuds, les arcs, les étiquettes et les indices des nœuds, les divisions en sous-réseaux communicatifs, etc. La transition entre deux représentations est donc elle-même le résultat d'un ensemble de transitions entre les éléments de ces représentations, c'est-à-dire un ensemble de **transitions élémentaires**.

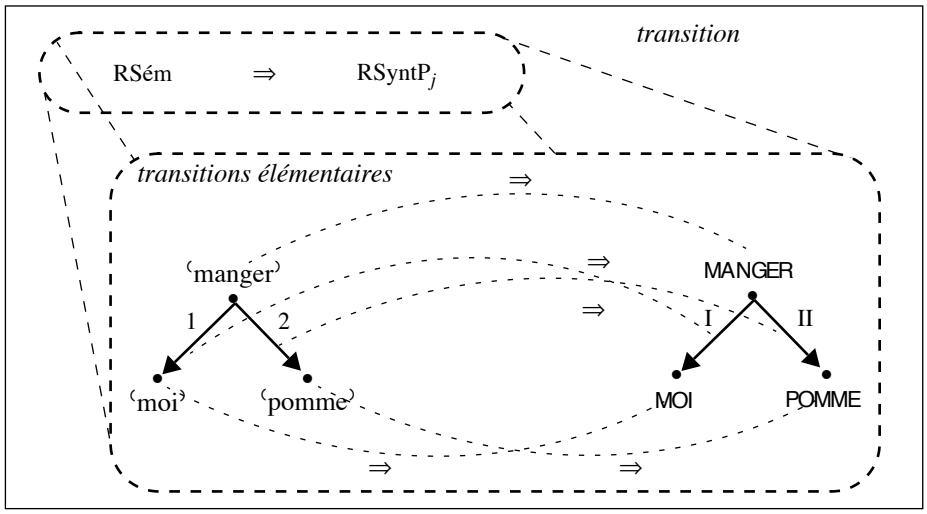


Figure 5. La transition vue comme un ensemble de transitions élémentaires

2. Cette affirmation vaut particulièrement pour la microsynthèse à des niveaux plus profonds, puisqu'il semble que plus on s'approche des niveaux de surface, moins les options sont nombreuses pour réaliser une représentation donnée.

Les transitions élémentaires s'effectuent par l'application de règles de transition. Ces règles seront présentées plus loin (*cf.* ch. III, § 2.1, p. 39). Il peut exister, pour un même élément d'une représentation source, plusieurs règles de transition permettant des transitions élémentaires concurrentes. Ainsi, un élément d'une source peut être réalisé de diverses façons. La réalisation d'un élément dans une ou plusieurs représentation(s) produit(s) s'appelle la **nanosynthèse**. La nanosynthèse n'est rien d'autre qu'un ensemble de transitions élémentaires concurrentes. On peut maintenant revoir la figure 4 en considérant cette fois la microsynthèse comme un ensemble de nanosynthèses. Ceci n'est aucunement en contradiction avec le fait que la microsynthèse est également le résultat d'un ensemble de transitions. La notion de nanosynthèse, que nous illustrons à la figure suivante, ne sera pas réutilisée dans ce mémoire. Nous l'introduisons néanmoins afin de compléter notre conceptualisation de la synthèse linguistique.

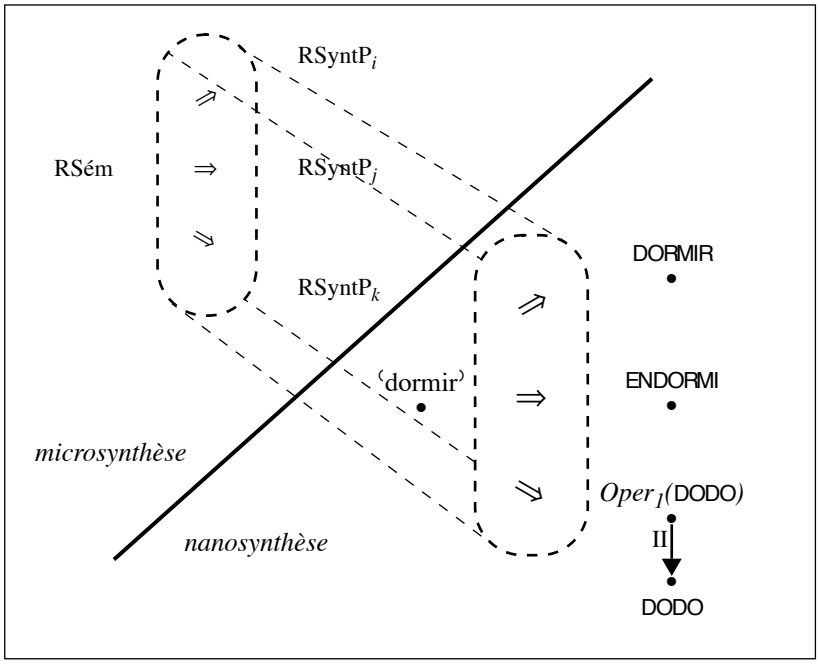


Figure 6. La microsynthèse vue comme un ensemble de nanosynthèses

Enfin, le modèle de la synthèse que nous présentons ici ne tient pas compte d'un autre type de règles, les règles d'équivalence, dont nous ne parlerons pas, si ce n'est que pour faire la remarque suivante. En théorie, avant de construire les représentations de niveau *N+1* correspondant à la représentation de niveau *N*, on

peut appliquer des règles d'équivalence pour dériver d'autres représentations de niveau *N*, pour ensuite effectuer autant de processus de synthèse parallèles à partir de ces nouvelles représentations. Du point de vue du traitement informatique, les règles d'équivalence ne posent pas de problème particulier et elles peuvent être traitées à peu près comme des règles de transition. Le lecteur pourra consulter notamment (Milićević, à paraître) et (Mel'čuk *et al.*, 1992: 9-58), qui traitent de ces règles d'équivalence.

Avant de voir de quoi sont faits les MST permettant la synthèse, nous proposons au chapitre II un survol du système *Sentence Garden*. Nous pourrons ainsi, au chapitre III, non seulement présenter les MST d'un point de vue théorique, mais également expliquer comment ils sont encodés dans ce système.

Chapitre II. Présentation de *Sentence Garden*

Les sections qui suivent présentent le système *Sentence Garden* dans son ensemble, sans trop entrer dans les détails et de façon plutôt informelle. Chaque composante importante du système sera revue plus en profondeur dans les chapitres III et IV.

Le système a été entièrement programmé en Prolog à l'aide de *SICStus Prolog*, version 3. Il ne fonctionne que sur la version classique de *MacOS*, mais l'existence de *SICStus Prolog* sur d'autres plateformes le rend facilement portable. Nous avons choisi ce compilateur justement pour la portabilité qu'il offrait, ainsi que pour son support de la programmation par objets. En effet, nous avons utilisé la programmation orientée objet pour la réalisation de ce système. Un **objet**, en informatique, est une structure de données permettant de décrire des classes dont on crée des instances. Chaque **classe** est décrite dans un programme en termes d'**attributs** (des variables associées aux instances de la classe) et de **méthodes** (des procédures permettant d'accéder aux attributs de la classe). Chaque **instance** créée possède les attributs et les méthodes de sa classe d'objets. Enfin, les classes peuvent avoir des sous-classes qui héritent de ses attributs et méthodes. La programmation orientée objet est présentée plus en détail dans (Delannoy, 1998). Bien que ce livre traite du langage C++, les principes de base de la programmation orientée objet qui y sont décrits sont valides également en Prolog.

1 Utilisation du système

Le but premier du linguiste utilisant *Sentence Garden* est de mettre à l'épreuve un MST d'une langue L afin d'en vérifier la validité. Il fournit au système son

MST ainsi qu'une RSém. Les RSém utilisées sont incomplètes : elles ne comprennent pas de structure rhétorique. Le synthétiseur tente alors de créer le plus de textes possible à partir de cette RSém en activant le MST fourni. La sortie du système doit être un ensemble de paraphrases ainsi que toutes les représentations intermédiaires créées lors du processus de synthèse. Toutefois, nous rappelons que la présente version du synthétiseur n'est qu'un prototype et ne permet pas de faire la synthèse au-delà du niveau syntaxique profond.

Le nom du système, *Sentence Garden*, est une métaphore du processus de synthèse qu'il effectue : la RSém fournie en entrée est vue comme une graine à partir de laquelle pousse une plante (le processus de synthèse lui-même, comme on le voit à la figure 2, p. 17) qui porte au bout de ses branches les fruits (les phrases) que récolte le linguiste.

En plus des phrases, le linguiste a accès à toutes les représentations intermédiaires qui ont permis de créer celles-ci. Pour chaque élément de chaque représentation, le système indique quelle règle du MST l'a produit. Ces informations permettent au linguiste de diagnostiquer divers problèmes dans son modèle. Ces problèmes éventuels peuvent être de deux types :

1) Il manque des règles au modèle. Dans ce cas, certaines représentations intermédiaires n'auront pas abouti à des phrases ou seront incomplètes.

2) Certaines règles sont erronées. Dans ce cas, certaines représentations intermédiaires (ainsi que les représentations et les phrases produites à partir de celles-ci) seront mal formées, comporteront des erreurs lexicales ou grammaticales, ou n'exprimeront pas le sens qu'encode la RSém de départ.

La figure de la page suivante illustre le processus de vérification d'un MST à l'aide de *Sentence Garden*.

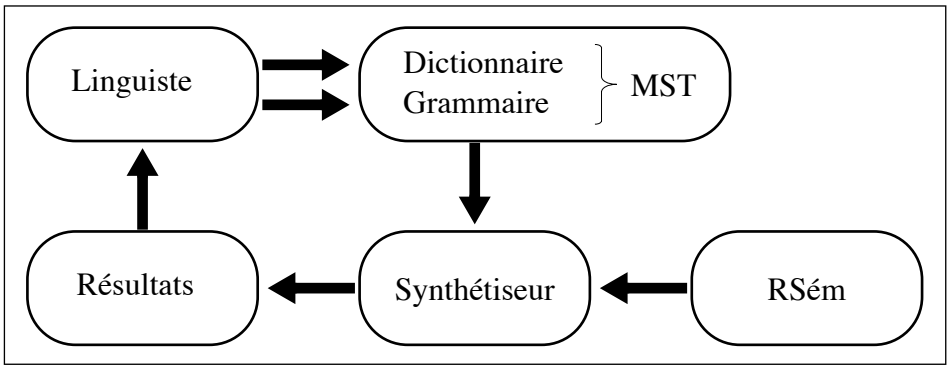


Figure 7. Utilisation de Sentence Garden

2 Les modules linguistiques

On aura compris que le synthétiseur lui-même ne contient aucune information propre à une langue précise. Tout ce qui relève de la description linguistique est consigné dans des fichiers bien distincts du synthétiseur comme tel. Ce dernier n'est ni plus ni moins qu'un moteur qui active les règles d'un MST. Cette division stricte entre descriptions linguistiques d'une part et procédures d'activation de ces descriptions linguistiques d'autre part, permet d'obtenir un système qui peut fonctionner pour n'importe quelle langue[1].

Bien que nous ne disposions pour l'instant que d'un MST (miniature!) du français, *Sentence Garden* a été conçu pour pouvoir accueillir un nombre illimité de modèles linguistiques. Il est possible de lancer un processus de synthèse dans la langue $\mathfrak{L}_1$, puis, quand celui-ci est terminé, de lancer une nouvelle synthèse dans la langue $\mathfrak{L}_2$ (le système purge alors la mémoire de toute information relative à $\mathfrak{L}_1$ pour charger le MST de $\mathfrak{L}_2$). Pour chaque langue $\mathfrak{L}$, on trouve un répertoire qui renferme tous les fichiers de son modèle.

1. En pratique toutefois, il y a un problème avec les divers systèmes d'écriture: *Sentence Garden* ne peut traiter que des langues utilisant un alphabet latin. Le problème provient en partie du dictionnaire utilisé, le *DiCo*: on y trouve des conventions d'écriture basées sur la distinction entre lettres majuscules et minuscules, alors que cette distinction n'existe pas dans tous les systèmes d'écriture. Nous n'avons toutefois pas cherché de solution à ce problème, puisqu'il n'est que technique et superficiel, et qu'il peut de toute façon être contourné en utilisant des transcriptions romanisées pour ces langues.

Un MST est composé principalement d'un dictionnaire et d'une grammaire. Ces deux composantes sont éditées à l'aide de logiciels appropriés offrant une interface facile à utiliser pour le linguiste. Elles sont par la suite importées par *Sentence Garden* et transformées sous la forme de fichiers Prolog directement utilisables par le synthétiseur. Ces deux composantes essentielles du MST seront décrites en détail au chapitre III. À celles-ci s'ajoutent un certain nombre d'informations qui ne sont pas encodables sous forme de règles ou d'articles de dictionnaire :

1) La hiérarchie des étiquettes sémantiques de L. Nous reparlerons plus loin de la notion d'étiquette sémantique (*cf.* ch. III, § 3.1, p. 44).
2) Des informations métalinguistiques à propos des traits de syntactique possibles en L. Ces informations sont utiles à la validation du dictionnaire lors de son importation par *Sentence Garden*, comme nous le verrons plus loin (*cf.* ch. III, § 3.4, p. 57), mais ne servent pas durant la synthèse.

3 Les RSém sources et les résultats de la synthèse

Sentence Garden en soi n'étant qu'un synthétiseur, on doit lui fournir des RSém déjà formées. Le système projeté devrait contenir un générateur de messages dont la tâche serait de créer de façon aléatoire des représentations sémantiques en respectant un certain nombre de règles afin d'en assurer la bonne formation. Un tel générateur aléatoire présente deux avantages :

1) Il peut générer des RSém auxquelles l'utilisateur n'aurait pas pensé.
2) Il génère rapidement un grand nombre de RSém.

Toutefois, dans la présente version de *Sentence Garden*, ce module n'est pas réalisé. Il a été remplacé par un programme qui fournit des RSém prédéfinies par l'utilisateur.

À chaque synthèse effectuée, un nouveau répertoire est créé sur le disque et les résultats de la synthèse y sont placés sous la forme de fichiers HTML. Cette méthode nous évite d'avoir à programmer une interface graphique tout en offrant une présentation plus conviviale que de simples fichiers au format texte : la mise en

forme permet de mieux représenter les données et la navigation à l'intérieur de ces données est facilitée par des liens hypertexte. Un fichier est créé pour chaque représentation qui a été produite au cours de la synthèse. Il y a également un fichier dans lequel se trouvent les articles de dictionnaire qui ont été utilisés ainsi qu'un autre contenant toutes les règles de la grammaire. Enfin, un fichier principal, nommé `harvest.html`, sert de point d'entrée à toutes ces informations. L'annexe 4 montre un exemple de « récolte », que nous allons commenter dans la section qui suit.

4 Un exemple complet de synthèse

Afin de bien illustrer comment s'utilise *Sentence Garden*, nous proposons un exemple complet. Il s'agit d'une expérience réelle effectuée avec notre système. À cette étape-ci de notre exposé, certaines parties de cet exemple paraîtront nébuleuses, puisque pour bien les comprendre, le lecteur aura besoin de notions qui n'ont pas encore été introduites. Toutefois, le but est seulement de donner une idée générale du fonctionnement du système, et pour cela il n'est pas nécessaire de comprendre tous les détails de ce qui suit.

Étape 1: Élaboration d'un MST

Le linguiste rédige des articles de dictionnaire. Ces articles sont stockés dans la base de données lexicales *DiCo*, éditée à l'aide du logiciel *FileMaker Pro*. L'annexe 1 montre un exemple d'article lexicographique, celui de REGRET**1**.

À l'aide du logiciel *JEditGraph*, dont nous reparlerons plus loin (*cf.* ch. III, § 2.2, p. 41), le linguiste développe également une grammaire. L'annexe 2 montre l'édition d'une règle avec ce logiciel. Pour notre exemple, nous avons préparé cinq règles simples que nous avons mises dans un format standard à l'annexe 3. Ces mêmes règles reviennent dans le fichier `grammar.html` de l'annexe 4, p. xvi. Encore une fois, nous insistons sur le fait qu'il n'est pas important pour l'instant de bien comprendre le formalisme de ces règles.

Étape 2: Construction d'une RSém

Le linguiste prépare une RSém qu'il souhaite utiliser afin de tester son modèle linguistique. Pour notre exemple, nous utiliserons la RSém suivante, dont une réalisation possible serait *La fière abeille regrette d'avoir abandonné le tigre*[1]:

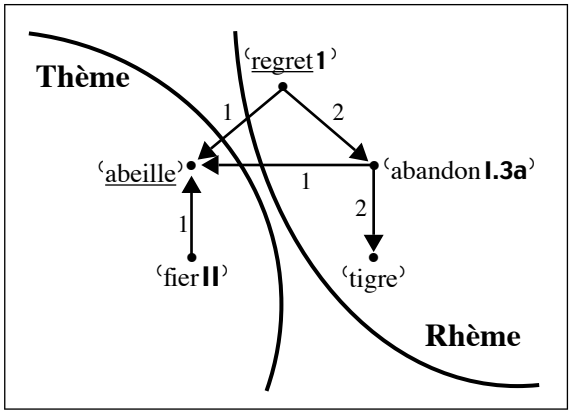


Figure 8. La RSém de départ

Cette RSém n'est que partielle, puisque les sens grammaticaux n'y figurent pas. Elle correspond à ce qui est affiché dans le fichier `semr5.html` de l'annexe 4, p.xviii.

Étape 3: Démarrage de *Sentence Garden*

Au démarrage du système, l'utilisateur voit défiler ce qui suit:

```
Loading the module file '/HD/users/francois/Sentence
Garden/Varieties/Français/Sem Rules'
Sem module rules successfully parsed
Creating <rule> objects:
     sag:1:i
     sag:1:attr
     sag:2:ii
     sag:2:attr
     sag-nod:1:oper1

Indexing the dictionary file
```

Ceci lui indique que les règles du module sémantique contenues dans les fichiers de *JEditGraph* ont bien été examinés et qu'elles étaient bien formées, et

1. L'absurdité de notre choix tient au nombre limité de lexies parmi lesquelles nous pouvions choisir: nous ne disposions que d'environ 300 lexies.

que cinq objets de la classe `règle` ont été créés (ces objets correspondent aux règles 3 à 7 de l'annexe 3). Viennent ensuite quelques instructions:

```
Welcome to the Sentence Garden
==============================

The default variety to be grown is: Français
To switch to another variety, submit
"variety( Language ).", where Language is the name of a
language as it appears in the 'Varieties' folder.

To start growing plants, submit "go." or "go( N ).",
where N is an integer in the range of 1 to 20.

If you need to import a dictionary or government pattern
file, call the Customs by typing "customs." (then,
follow the instructions).
```

L'utilisateur n'a plus qu'à taper `go.` pour lancer la synthèse. Il voit alors défiler ceci:

```
Growing the seed semr5
Retrieving 'TIGRE'
Retrieving 'ABANDON#I.3a'
Retrieving 'ABANDON#0'
Retrieving 'REGRET#1'
Retrieving 'REGRET#0'
Retrieving 'FIER#II'
Retrieving 'FIER#0'
Retrieving 'ABEILLE'
Plant # 1 harvested
```

Dans la première ligne, `semr5` est le nom de l'objet qui correspond à la RSém que l'utilisateur a fournie en entrée. Les huit lignes suivantes apparaissent à mesure que le synthétiseur consulte les articles de dictionnaire dont il aura besoin pour effectuer la synthèse. Finalement, une fois le processus terminé, la dernière ligne apparaît, indiquant qu'il ne reste plus qu'à aller savourer ses fruits…

Étape 4: Examen des résultats et diagnostic

À la fin du processus, sept fichiers ont été créés:

```
HARVEST.html
dictionary.html
grammar.html
semr5.html
```

```
dsyntr35.html
dsyntr48.html
dsyntr53.html
```

L'annexe 4 montre (au moins partiellement) le contenu de tous ces fichiers. Le fichier principal est `harvest.html`. Il est le point d'entrée des données. On y trouve des pointeurs vers les autres fichiers, et on voit en un coup d'œil quelles représentations sont les produits de quelles sources.

Pour des raisons d'espace, il n'est pas possible de montrer tout le fichier `dictionary.html` dans l'annexe 4, mais dans le fichier original on peut voir que seuls les articles de dictionnaire utiles à la synthèse ont été chargés en mémoire.

Dans le fichier `grammar.html` on voit que, en plus des cinq règles que nous avions créées à l'aide de *JEditGraph*, d'autres règles ont été créées automatiquement lors du chargement des articles de dictionnaire. Nous reviendrons plus loin sur la création automatique de ces règles lexicales (*cf.* ch. III, § 3.5, p. 60).

Le fichier `semr5.html` montre la RSém de départ utilisée lors de cette synthèse (elle correspond à la RSém de la figure 8, p. 28). Les fichiers `dsyntr35.html`, `dsyntr48.html` et `dsyntr53.html`, quant à eux, présentent les RSyntP produites à partir de la RSém de départ. Notons que la RSyntP du fichier `dsyntr35.html` n'est pas celle d'une phrase, mais bien d'un groupe nominal. On remarque également que notre format de sortie ne montre pas la structure anaphorique. Pour chaque produit, le système fournit la liste de toutes les règles ayant permis chaque transition élémentaire, ainsi que la correspondance entre les nœuds de la source et ceux du produit. En examinant ces informations pour les produits obtenus et en effectuant d'autres synthèses à partir de RSém différentes, le linguiste peut identifier et corriger les erreurs dans son modèle linguistique.

Notons finalement que les étapes présentées ci-dessus correspondent aux étapes d'une démarche scientifique. On formule d'abord une hypothèse (construction

d'un MST) puis on procède à un certain nombre d'expériences qui permettent de confirmer ou d'infirmer cette hypothèse. À la lumière des résultats obtenus, on peut formuler une nouvelle hypothèse et reprendre le processus. *Sentence Garden* peut donc être vu comme un laboratoire linguistique qui permet aux linguistes de mettre à l'épreuve leurs hypothèses, conférant ainsi une valeur scientifique à leur travail. Puisque le synthétiseur lui-même ne contient aucune description linguistique, il constitue une sorte de « laboratoire d'essais générique », permettant de vérifier des modèles linguistiques de langues très variées. Nous allons maintenant voir quelle forme exactement prennent ces modèles.

Chapitre III. Les modèles Sens-Texte

Comme nous l'avons mentionné au chapitre précédent, le moteur de synthèse en tant que tel ne contient aucune information de nature linguistique. La description des langues se trouve entièrement confinée à l'intérieur des modules linguistiques. Néanmoins, pour pouvoir activer ces modèles linguistiques, le moteur de synthèse doit connaître le métalangage utilisé pour les écrire. Le métalangage est celui normalement utilisé dans le cadre de la TST. Il est décrit dans *Sentence Garden* en termes de classes d'objets. Nous proposons de faire dans ce chapitre une revue du métalangage Sens-Texte, à la fois d'un point de vue théorique et pratique (son encodage dans *Sentence Garden*).

Un modèle Sens-Texte de la langue $\mathcal{L}$ est constitué d'une grammaire et d'un dictionnaire. C'est un dispositif logique qui établit la correspondance entre les sens et les textes de $\mathcal{L}$. Plus précisément, il établit la correspondance entre la ***représentation*** des sens et la ***représentation*** des textes, via un ensemble de représentations intermédiaires. Ainsi, ce qu'un MST « manipule », ce sont des représentations linguistiques de divers niveaux (sémantique, syntaxique, morphologique et phonologique). Nous commencerons donc par décrire le formalisme de ces représentations linguistiques, pour ensuite nous attaquer à la grammaire puis au dictionnaire.

1 Les représentations linguistiques

1.1 Caractérisation formelle des représentations linguistiques en TST

Les représentations linguistiques utilisées dans le cadre de la TST sont avant tout des graphes composés de nœuds et d'arcs. Nous allons décrire chaque élément nécessaire à la construction de telles représentations.

Les nœuds

Un **nœud** sert de support formel à ***1)*** une **étiquette**, c'est-à-dire un sémantème, une lexie, un morphème ou un phonème, selon le niveau de représentation, et ***2)*** un **indice**, soit une liste potentiellement vide de grammèmes et de traits de syntactique (= combinatoire) qui sont associés à l'étiquette du nœud.

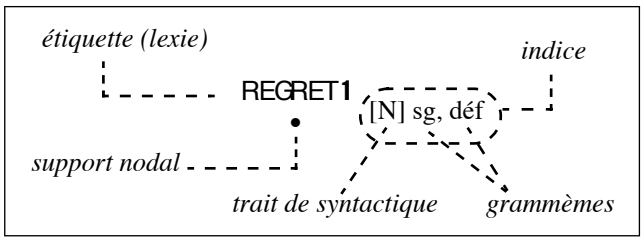


Figure 9. Un nœud (syntaxique profond)

Les arcs

Un **arc** (toujours orienté) relie deux nœuds dans une représentation. Aux niveaux sémantique et syntaxique (profond et de surface), chaque arc est étiqueté d'un nom de relation. Les relations sémantiques (1 à 6) et syntaxiques profondes (I à VI, ATTR, COORD et APPEND) sont universelles, alors que les relations syntaxiques de surface varient d'une langue à l'autre[1].

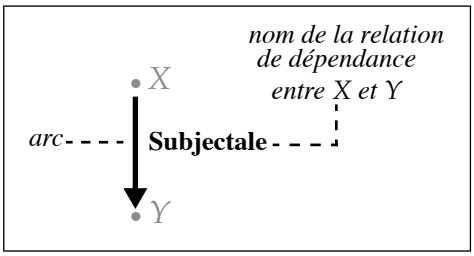


Figure 10. Un arc (syntaxique de surface)

Les représentations sémantiques

Une RSém est constituée de trois structures: sémantique [SSém], sémantico-communicative [SSém-Comm] et sémantico-rhétorique [SSém-Rhét]. On sait encore très peu de choses sur la SSém-Rhét et sur sa formalisation; pour cette raison, nous l'ignorerons dans notre exposé. La structure porteuse, la SSém, est un réseau connexe et acyclique constitué de nœuds étiquetés de sémantèmes et d'arcs étiquetés de relations sémantiques (1 à 6). Pour une étude plus pointue des réseaux sémantiques, le lecteur pourra consulter notamment (Polguère, 1997a). La SSém-Comm est *grosso modo* une division du réseau en sous-réseaux communicatifs. Pour chaque sous-réseau, un nœud est identifié comme en étant le nœud dominant (nous indiquons ce nœud par un soulignement dans les figures). Enfin, ces sous-résaux sont tous étiquetés d'une des valeurs des 8 oppositions communicatives suivantes:

1) rhème ~ thème ~ spécificateur

2) donné ~ nouveau

3) focalisé ~ non-focalisé

4) avant-plan ~ arrière-plan

5) emphatisé ~ non-emphatisé

6) présupposé ~ posé

7) unitaire ~ articulé

8) signalé ~ performatif ~ communiqué

1. Voir (Iordanskaja & Mel'čuk, 2000) pour une présentation de la méthode à suivre pour établir l'inventaire des relations syntaxiques de surface d'une langue, et (Mel'čuk, à paraître), sur les types de dépendances linguistiques.

Nous ne pouvons pas présenter chacune de ces oppositions, et nous renvoyons le lecteur à (Mel'čuk, 2001) pour une étude en profondeur de ces notions.

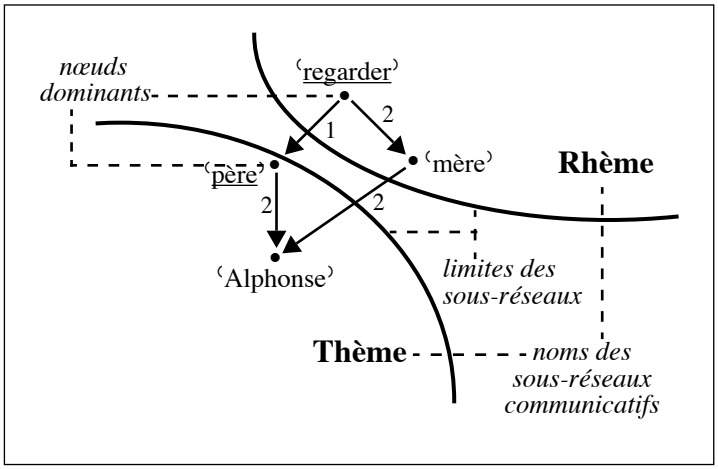


Figure 11. Une représentation sémantique

Les représentations syntaxiques profondes

Comme les RSém, les RSyntP sont composées de trois structures: syntaxique profonde [SSyntP], syntaxico-communicative profonde [SSynt-CommP] et syntaxico-anaphorique profonde [SSynt-AnaphP]. La structure porteuse, la SSyntP, est un arbre de dépendance, ce qui signifie que tous les nœuds sauf un, appelé le **sommet syntaxique**, ont un et un seul gouverneur (le sommet syntaxique n'ayant aucun gouverneur). Les nœuds sont tous étiquetés de lexèmes ou de fonctions lexicales et indicés de grammèmes; les arcs sont étiquetés de relations syntaxiques profondes (I à VI, ATTR, COORD et APPEND). Les seuls lexèmes et grammèmes apparaissant au niveau syntaxique profond sont ceux qui sont sémantiquement pleins, c'est-à-dire ceux qui ont un correspondant direct au niveau sémantique (sont donc exclus les prépositions régies, les grammèmes d'accord, etc.). La SSynt-AnaphP indique l'anaphore entre des nœuds syntaxiques correspondant au même nœud sémantique. Les anaphores sont indiquées dans les figures au moyen d'un arc bidirectionnel en pointillés. Enfin, peu de recherches ont été faites sur la SSynt-CommP et sa formalisation, aussi n'en parlerons-nous pas.

Pour la même raison, *Sentence Garden* ne gère pas la construction de la structure communicative au niveau syntaxique.

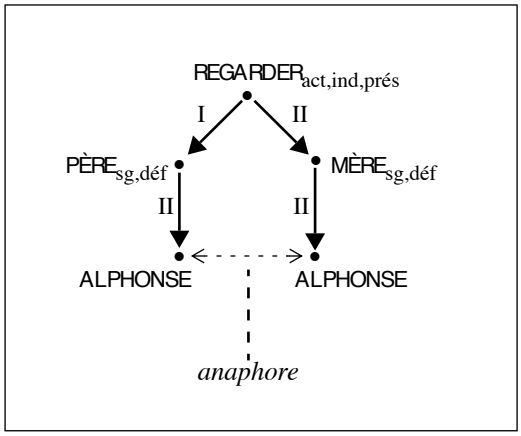


Figure 12. Une représentation syntaxique profonde (partielle)

Puisque *Sentence Garden* n'effectue que la première microsynthèse, nous ne décrirons pas les représentations linguistiques utilisées aux niveaux syntaxique de surface, morphologique et phonologique. Le lecteur pourra consulter (Mel'čuk, 1997) pour une caractérisation formelle des représentations utilisées à ces niveaux.

1.2 L'encodage des représentations linguistiques dans *Sentence Garden*

Le langage formel des représentations linguistiques est décrit dans notre système en termes de classes d'objets informatiques (*cf.* ch. II, page 23). Cette section présente les classes d'objets qui ont été utilisées.

Les nœuds

Un objet de la classe `nœud` possède quatre attributs :

1) Une étiquette.
2) Une liste d'indices.
3) Une liste d'arcs entrants.
4) Une liste d'arcs sortants.

Les arcs

Un objet de la classe `arc` a également quatre attributs :

1) Une étiquette.
2) Un type (*dépendance* ou *anaphore*). Les objets encodant les représentations syntaxiques profondes ne contiennent pas de structure anaphorique comme telle. Cette information sera encodée par le type d'arc entre les nœuds, et c'est pourquoi nous avons besoin de cet attribut supplémentaire dans la classe d'objets `arc`.
3) Le nœud qui se trouve au « début » de l'arc (le nœud mère).
4) Le nœud qui se trouve à la « fin » de l'arc (le nœud fille).

Les graphes

La classe `graphe` ne correspond à aucune représentation linguistique dans la TST. C'est une classe dont les objets encodant les représentations linguistiques sont des types particuliers. Un graphe possède dix attributs :

1) Une liste de nœuds.
2) Une liste d'arcs de type *dépendance*.
3) Une liste de sous-graphes communicatifs. Cet attribut correspond à la structure communicative dans la TST. Un sous-graphe communicatif est encodé par une liste dont le premier élément est le nom d'un sous-réseau communicatif, le second son nœud dominant, et les autres éléments sont les autres nœuds faisant partie de ce sous-réseau. À l'intérieur de cette liste peuvent être imbriquées d'autres listes correspondant à des sous-réseaux communicatifs enchâssés. Par exemple, supposons une RSém comprenant les nœuds *a*, *b*, *c* et *d* répartis comme suit : le thème n'a que le nœud *a* (qui est donc son nœud dominant), le rhème contient tous les autres nœuds (*b* en étant le nœud dominant) et comprend également un sous-réseau mis en avant-plan composé du nœud *d* seulement. Alors, cette structure communicative serait encodée comme suit :

   ```
   [[ thème,a],[ rhème,b,c,[ avant_plan,d]]].
   ```

4) L'identificateur du processus de synthèse dont le graphe fait partie.
5) La représentation source du graphe.
6) La liste des représentations produits du graphe.

7) Une liste de correspondances indiquant, pour chaque élément du graphe, à partir de quel élément de la représentation source et grâce à quelle règle linguistique il a été créé.

8) Une liste de correspondances nodales indiquant pour chaque nœud du graphe le nœud de la représentation source dont il est le correspondant.

9) La liste des nœuds de la représentation source qui ont été « revisités » lors du parcours de la source. Nous verrons plus loin de quoi il s'agit exactement (*cf.* ch.IV, §2, p.64). Pour l'instant, nous nous contenterons de dire que cette information équivaut à spécifier quels sont les nœuds de la source qui ont plus d'un correspondant dans le graphe en question.

10) La liste des éléments de la représentation source qui n'ont pas été réalisés dans le graphe.

La classe `graphe` est munie d'une méthode permettant de vérifier la bonne formation des instances créées. Leur bonne formation est vérifiée simplement en s'assurant que ***1)*** il existe un chemin entre un nœud et tous les autres nœuds du graphe (le graphe est connexe) et ***2)*** il n'existe aucun chemin strictement ascendant ou strictement descendant entre un nœud et lui-même (le graphe est acyclique). Un graphe est la structure porteuse de toute représentation linguistique. Par exemple, une SSém, structure porteuse d'une RSém, est un réseau.

Les arbres

La classe `arbre` hérite de la classe `graphe`. Les règles de bonne formation d'un arbre sont toutefois différentes de celle d'un graphe : un arbre doit ***1)*** être un graphe bien formé (connexe et acyclique), ***2)*** n'avoir qu'un seul nœud qui n'ait aucun gouverneur, et ***3)*** n'avoir aucun nœud ayant plus d'un gouverneur.

Les représentations sémantiques

La classe d'objets `rsém` hérite également de la classe `graphe`. Cette classe comprend une méthode qui retourne les différents parcours possibles pour une instance donnée. Nous verrons plus loin de quoi il s'agit (*cf.* ch.IV, §2, p.64).

Les représentations syntaxiques profondes

Enfin, la classe `rsyntp` hérite de la classe `arbre` (et donc indirectement de la classe `graphe`). Elle comprend, en plus des attributs hérités des classes `graphe` et `arbre`, un attribut dont la valeur est une liste d'arcs de type *anaphore* qui encode la SSynt-AnaphP.

2 La grammaire

2.1 Caractérisation formelle des grammaires Sens-Texte

Un MST est un ensemble de règles établissant la correspondance entre des sens et des textes. Les MST sont principalement composés d'un dictionnaire et d'une grammaire. La frontière entre les deux est somme toute assez mince. La différence entre une règle grammaticale et une règle lexicale réside uniquement dans le type de signe manipulé par les règles. Nous ne pouvons évidemment pas nous lancer dans une typologie des signes linguistiques — pour cela, le lecteur pourra consulter (Mel'čuk, 1993) —, ce qui serait nécessaire à une véritable définition de *grammaire* et de *dictionnaire*, mais nous dirons la chose suivante. Une règle qui ne manipule que des relations entre des nœuds ou encore des signes appartenant à des ensembles petits et fermés de signes (des grammèmes) est une règle grammaticale. Les règles lexicales, elles, manipulent des signes lexicaux, ou encore des classes de signes lexicaux (par exemple, une règle pourrait viser toutes les lexies ayant une même étiquette sémantique).

Un MST établit la correspondance entre des sens et des textes en passant par un ensemble de représentations intermédiaires. Un MST est donc composé de modules distincts, chacun assurant la correspondance entre des représentations de niveaux adjacents. Il en va de même pour la grammaire. Le linguiste qui développe une grammaire doit donc en fait développer un ensemble de modules séparés les uns des autres, mais interdépendants. Dans le cadre de *Sentence Garden* toutefois, seul le module sémantique (le module établissant la correspondance entre les

RSém et les RSyntP) est pris en charge, puisque seule la première microsynthèse est effectuée.

Un **module** n'est rien d'autre qu'un ensemble de règles permettant une microsynthèse[1]. Une **règle de transition** est la mise en correspondance, sous certaines conditions, de deux représentations élémentaires[2] de niveaux adjacents[3]: $R_1 \Leftrightarrow R_2$ | *Conditions*. Dans le cas d'une règle du module sémantique, les éléments pouvant apparaître dans R_1, que nous appelons la **partie gauche** de la règle, sont ceux que l'on retrouve dans une RSém. Les éléments apparaissant dans R_2, la **partie droite**, sont ceux d'une RSyntP. Les **conditions** sont exprimées dans une proposition logique. L'annexe 3 montre quelques exemples de règles de grammaire (hypothétiques) du module sémantique. La notation *r(X)* dans la partie droite des règles correspond à ce qu'on note souvent X' dans la littérature Sens-Texte et sert à désigner le nœud ou le sous-graphe correspondant au nœud X de la partie gauche de la règle[4].

De telles règles, lorsqu'elles sont appliquées (en synthèse) à une représentation de niveau *N*, permettent la construction de la représentation de niveau *N+1* correspondante. Par exemple, lorsqu'une règle est appliquée à une partie d'une RSém, elle permet de construire la partie correspondante dans le produit de cette RSém, une RSyntP. Les règles correspondent donc à une transition élémentaire (*cf.* ch.I, §4, p.17).

Dans une règle, les éléments n'ont pas tous le même statut. Il y a des éléments actifs et des éléments contextuels. Un **élément actif** (marqué en gras) est un élément pour lequel la règle établit effectivement une correspondance. Les

1. Nous nous plaçons uniquement du point de vue de la synthèse, bien qu'un module soit en théorie bidirectionnel et permette autant l'analyse que la synthèse.
2. À notre connaissance, aucune étude n'a été faite sur la nécessité ou non pour ces représentations élémentaires d'être connexes. Toutefois, *Sentence Garden* ne peut traiter que des règles dont les graphes élémentaires sont connexes.
3. Encore une fois, nous faisons abstraction des règles d'équivalence et ne parlons que des règles de transition (*cf.* ch.I, §4, p.17).
4. Le *r* vient de l'anglais *right*.

éléments contextuels ne sont pas mis en correspondance par la règle, et ils sont équivalents à des conditions.

Enfin, les règles utilisées dans *Sentence Garden* tiennent compte de la hiérarchie communicative entre les nœuds. Lorsqu'un nœud est souligné dans la partie gauche ou droite d'une règle, cela signifie qu'il domine communicativement les autres nœuds de la même partie de cette règle. La notion de hiérarchie communicative n'ayant pas encore été introduite, nous prions le lecteur de bien vouloir patienter jusqu'au prochain chapitre, où nous tenterons d'expliquer cette notion (*cf.* ch.IV, §2, p.64). La figure suivante montre un exemple de règle. Il s'agit en fait de la règle 7 de l'annexe 3.

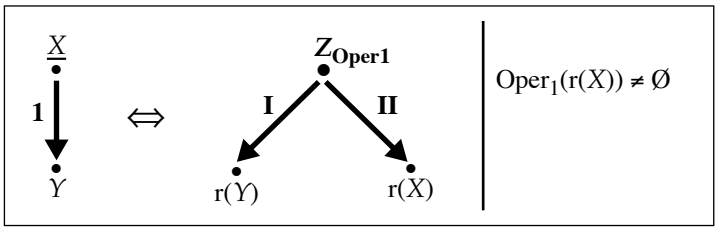


Figure 13. Un exemple de règle de grammaire

2.2 L'encodage de la grammaire dans *Sentence Garden*

Dans le cadre de *Sentence Garden*, les règles de la grammaire sont éditées avec un logiciel fait sur mesure par Jasmin Lapalme, *JEditGraph*. Ce logiciel offre un environnement graphique simple à utiliser. L'annexe 2 montre une sortie d'écran prise lors de l'édition d'une règle sémantique. Une interface graphique offre un double avantage: ***1)*** le développement de la grammaire par le linguiste est beaucoup plus « convivial » et ***2)*** puisque l'utilisateur humain ne voit pas l'encodage sous-jacent, ce dernier peut être extrêmement formalisé (au point d'en être pratiquement illisible par un humain normalement constitué), ce qui le rend d'autant plus facile à lire par un programme informatique. Le système *MATE* offre un environnement graphique encore plus convivial et plus puissant que celui utilisé dans *Sentence Garden*. Toutefois, cette interface graphique n'était pas encore développée au moment de commencer le développement de notre système.

Chaque module créé à l'aide de *JEditGraph* est sauvegardé dans un fichier séparé. Le format de ces fichiers est semblable au langage XML sans en être tout à fait. Au moment du démarrage de *Sentence Garden*, ces fichiers sont lus par le système et les règles sont alors encodées sous forme d'objets. Les objets nécessaires à l'encodage d'une grammaire sont les suivants.

Les règles

Un objet de la classe `règle` possède six attributs :

1) La liste de tous les éléments (nœuds, indices des nœuds, arcs, etc.) de la partie gauche de la règle.
2) La liste des éléments actifs à gauche.
3) La liste de tous les éléments de la partie droite de la règle.
4) La liste des éléments actifs à droite.
5) Des conditions, exprimées sous la forme d'un terme Prolog complexe.
6) Une liste de règles qui doivent être appliquées si la présente règle s'applique.[1]

Les modules linguistiques

La classe `module` est une classe dont aucune instance ne sera créée, mais dont héritent d'autres classes. Elle a cinq attributs :

1) La liste de toutes les règles de transition du module.
2) La liste de toutes les règles d'équivalence du module.
3) Une instance d'une sous-classe de la classe `représentation`. Cet attribut est utilisé par le système pour mémoriser quelle représentation est la source sur laquelle il est en train de travailler à un instant donné.
4) Une autre instance d'une sous-classe de la classe `représentation`. Cet attribut est utilisé par le système pour mémoriser quelle représentation est la cible sur laquelle il est en train de travailler à un instant donné.
5) Une liste de couples servant à garder la trace des matchings (*cf.* ch. IV, § 4, p. 72) effectués entre les éléments d'une règle et ceux d'une

1. Cet attribut n'est pas utilisé dans le prototype actuel.

représentation. Tout comme les deux précédents, cet attribut est utilisé par le système au moment de la synthèse.

Deux remarques s'imposent. D'abord, les trois derniers attributs peuvent surprendre. S'ils sont placés dans la classe `module`, c'est que c'est dans cette classe que sont définies les méthodes effectuant le majeure partie du travail de synthèse ; elle est en fait le cœur du synthétiseur. Nous ne décrirons pas ici ces méthodes, mais que le lecteur se console : nous proposons d'y consacrer à peu près la moitié du prochain chapitre (*cf.* ch. IV, § 3, p. 71, et les sections subséquentes).

Enfin, les règles d'équivalence, bien qu'elles existent dans *Sentence Garden* et qu'elles fassent partie des modules, ne sont pas appliquées dans la version actuelle du système. Toutefois, tout est déjà en place pour qu'elles le soient, et il ne faudrait que quelques modifications au programme pour que le synthétiseur tienne compte également de ces règles.

Les modules sémantiques

La classe `Sém` est une sous-classe de la classe `module`. En plus des attributs hérités de sa super-classe, elle possède deux attributs qui lui sont propres :

1) Le niveau du module (la valeur de cet attribut est *Sém*).

2) Le niveau du module suivant (la valeur de cet attribut est *SyntP*).

C'est une instance de cette classe qui est créée lorsque le module sémantique d'une langue est chargé.

La grammaire

La classe `grammaire` est une classe n'ayant aucun attribut. Y sont définies des méthodes servant à gérer le chargement des modules linguistiques ainsi que l'affichage des règles dans les fichiers de sortie.

3 Le dictionnaire

Le dictionnaire utilisé par *Sentence Garden* est le *Dictionnaire des dérivations et collocations* [*DiCo*], qui est développé depuis quelques années à l'Université de Montréal par les professeurs Mel'čuk et Polguère ainsi que leurs étudiants. Il s'agit d'une version électronique et allégée[1] du *Dictionnaire explicatif et combinatoire* [*DEC*]. Il n'existe pour l'instant qu'une version française du *DiCo*, ce qui explique notre choix de langue pour tester *Sentence Garden*. Toutefois, il n'est pas exclu qu'un jour naissent d'autres versions de ce dictionnaire[2]. Notons finalement que du *DiCo* est dérivé un dictionnaire grand public, le *Lexique actif du français* [*LAF*], dont la publication ne devrait plus tellement tarder.

La lexicographie ayant toujours occupé une place centrale dans le cadre de la TST, de nombreuses publications traitent de ce sujet. Une simple revue de la littérature existante prendrait probablement tout un chapitre, aussi nous contenterons-nous de donner quelques références incontournables. Le *DiCo* a été décrit dans (Polguère, 2000a). Son grand frère, le *DEC*, est présenté dans le volume 1 du dictionnaire lui-même (Mel'čuk *et al.*, 1984). Pour sa part, (Mel'čuk *et al.*, 1995) traite à la fois du *DEC* et du *DiCo*. Enfin, le *LAF* est présenté dans (Polguère, 2000b). Nous proposons tout de même une brève caractérisation formelle du *DiCo* ainsi qu'une discussion de son utilisation dans *Sentence Garden*.

3.1 Caractérisation formelle du *DiCo*

Le *DiCo* est une base de données [BD] sur support électronique, éditée au moyen du logiciel *FrameMaker Pro*. Elle est composée de fiches, elles-mêmes séparées en champs. Chaque fiche contient la description d'une lexie. À chaque champ correspond une catégorie d'informations lexicales. L'annexe 1 montre la

1. Le *DiCo* ne contient pas de définitions, contrairement au *DEC*. Toutefois, il contient un plus grand nombre de vocables.
2. Il existe notamment *Papillon*, un projet international de dictionnaire électronique multilingue composé de plusieurs dictionnaires monolingues du format du *DiCo* mis en correspondance. Ce projet est présenté dans (Mangeot-Lerebours, 2000).

fiche du lexème REGRET**1**, exemple auquel nous référerons tout au long de la présente section.

Les noms des lexies

Le champ *nom* sert à écrire le nom du superarticle de dictionnaire dont l'article fait partie. Il s'agit en fait du nom du vocable auquel appartient la lexie. Dans le cas de vocables homonymiques, par exemple PALAIS^{1} (celui que l'on a dans la bouche) *vs*. PALAIS^{2} (celui dans lequel plusieurs aimeraient habiter), ils sont distingués dans le *DiCo* par un numéro entre parenthèses: `palais(1)` et `palais(2)`. Par ailleurs, lorsqu'on a deux vocables homonymiques n'ayant pas la même partie du discours, ils sont distingués par l'ajout d'un trait suivi de la partie du discours. Par exemple: `amoureux-adj` *vs* `amoureux-n`.

Les numéros d'identification des lexies

Le champ suivant (juste à droite dans la figure en annexe) indique le numéro d'identification de la lexie à l'intérieur du vocable. La numérotation des lexies d'un même vocable reflète les différences sémantiques entre elles: les chiffres romains sont utilisés pour des différences assez grandes, les chiffres arabes pour des différences moins fortes, et finalement des lettres pour des nuances de sens très fines ou régulières (*cf.* Mel'čuk *et al.*, 1995: 162-171). Pour chaque vocable polysémique, il y a une fiche, numérotée *0* (zéro), dans laquelle est consignée l'information commune à toutes les lexies du vocable. Par exemple, c'est typiquement dans cette fiche qu'on indique la partie du discours des lexies du vocable, puisqu'en général les lexies d'un même vocable appartiennent à la même partie du discours. L'information inscrite dans cette fiche est héritée par toutes les lexies du vocable, à moins d'indication contraire.

Le syntactique des lexies

C'est dans le champ *cg* (= *caractéristiques grammaticales*) que sont consignées les informations sur le syntactique de la lexie (qu'on appelle également sa *combinatoire*) autres que le régime et les fonctions lexicales, qui ont chacun leur

propre champ. On note ici la partie du discours de la lexie, le fait qu'elle puisse ou non se combiner avec tel ou tel grammème, etc. Pour toute lexie, il est obligatoire d'en indiquer partie du discours, soit dans sa propre fiche, soit dans celle du vocable. Les autres traits de syntactique pouvant apparaître dans ce champ sont tous facultatifs. On remarquera que dans notre exemple en annexe, la lexie REGRET**1** n'a aucune indication quant à sa partie du discours dans le champ *cg*, puisque cette information se trouve dans la fiche REGRET**0** et qu'elle est héritée par toutes les lexies du vocable.

La description sémantique des lexies

Pour sa part, le champ *cs* (= *caractéristiques sémantiques*) est une description sémantique de la lexie. Nous disons bien *description sémantique* et non *définition*, puisqu'en fait le *DiCo* ne contient pas de véritables définitions, en ce sens qu'il ne propose pas une ***décomposition*** sémantique des lexies décrites. Ce qu'encode le champ *cs*, c'est: ***1)*** la **structure actancielle** des lexies (c'est-à-dire le nombre d'actants sémantiques qu'elles contrôlent), et ***2)*** les étiquettes sémantiques des lexies et de leurs actants sémantiques.

L'**étiquette sémantique** d'une lexie *L* est la composante centrale de la définition de *L*. Les étiquettes sémantiques utilisées pour décrire les lexies d'une langue **L** sont choisies parmi une liste hiérarchisée des étiquettes sémantiques de **L**. Cette liste varie d'une langue à l'autre et doit être établie de façon empirique pour chaque langue. La hiérarchie des étiquettes sémantiques de **L** n'est pas une ontologie. Ce n'est pas une classification des ***choses*** du monde réel, mais bien une hiérarchie de ***sens*** de **L** qui sont en relation d'inclusion les uns par rapport aux autres. Enfin, ceci n'est qu'une caricature de la notion d'étiquette sémantique; pour une présentation détaillée de ce concept, voir (Milićević, 1997).

Nous reproduisons ici le champ *cs* pour la lexie REGRET**1** de l'annexe 1:

```
sentiment: ~ DE LA personne X À CAUSE DE SON action Y(X)
```

Cela signifie que l'étiquette sémantique de REGRET**1** est *sentiment*, que la lexie contrôle deux actants sémantiques *X* et *Y*, *Y* ayant lui-même *X* comme actant

(*cf.* `Y(X)`), dont les étiquettes sémantiques sont respectivement *personne* et *action*. Les mots en majuscules ne font pas partie de la description sémantique. Ils servent à faciliter la lecture de l'article par un humain et apparaissent dans le *LAF*.

Les tableaux de régime

En plus d'actants sémantiques [ActSém], une lexie contrôle des actants syntaxiques profonds [ActSyntP] et des actants syntaxiques de surface [ActSyntS]. La correspondance entre les actants d'une lexie à ces trois niveaux de représentation n'est pas toujours régulière, c'est pourquoi cette information doit être indiquée dans le dictionnaire pour chaque lexie. C'est dans le champ *tr* (= *tableau de régime*) qu'on inscrit le régime de la lexie (qu'on appelle aussi sa *valence active*). Par exemple, le champ *tr* contient, pour la lexie REGRET**1** :

```
X = I = de N, A-poss
Y = II = _à cause de_ N, _à propos de_ N, de V-inf
```

Chaque ligne dans le tableau correspond à un ActSém. Les variables *X*, *Y*, …, ne signifient pas *ActSém 1*, *2*, etc. Ces variables renvoient plutôt à celles utilisées dans la description sémantique de la lexie et pourraient apparaître dans n'importe quel ordre (bien que normalement la variable *X* soit utilisée pour l'ActSém 1, *Y* pour le 2, et ainsi de suite). Ainsi, le tableau ci-haut signifie que le premier ActSém (première ligne) correspond au *X* du champ *cs* (donc celui qui porte l'étiquette sémantique *personne* dans notre exemple), qu'il doit être réalisé en syntaxe profonde par le premier ActSyntP (`I`), et qu'il peut se réaliser en surface soit par un nom précédé de la préposition DE (`de N`) soit par un adjectif possessif (`A-poss`). Le symbole « `_…_` » sert à marquer les phrasèmes.

Les fonctions lexicales

Finalement, le champ *fl* (= *fonctions lexicales*) donne la liste des fonctions lexicales [FL] de la lexie, ainsi que les valeurs de ces fonctions. Le formalisme utilisé est assez complexe et ne sera pas décrit ici en détail. *Grosso modo*, on trouve obligatoirement pour chaque FL son nom entre accolades suivi de sa valeur, dont les éléments sont séparés par une virgule (éventuellement un point-virgule dans le

cas de nuances sémantiques entre les éléments de valeur). Les noms des FL peuvent être précédés de noms vulgarisés, en langue commune contrôlée, placés entre les balises `/*` et `*/`. Il s'agit des noms de FL qui seront utilisés dans le *LAF*. Par exemple, toujours pour REGRET**1** :

```
/*Intense*/
{Magn} grand, profond, vif | antépos < extrême, infini;
       amer, cruel, cuisant
```

Cela signifie que dans le contexte de la lexie REGRET**1** la FL *Magn* peut être paraphrasée, pour ainsi dire, par l'expression *intense*. Dans le *LAF*, le nom « scientifique » de la FL n'apparaîtra jamais, il sera remplacé par ce nom vulgarisé, ce qui en facilite la consultation pour des utilisateurs non initiés à la formalisation des FL (qu'on peut supposer nombreux).

Les éléments de valeur des FL peuvent également être suivis de leur régime, entre crochets. Par exemple, encore pour REGRET**1**, on trouve la FL suivante (le gras est de nous):

```
{Oper21} causer [ART ~ à/chez N=X], laisser [des ~s à
         N=X], provoquer [du/des ~(s) chez N=X]
```

L'expression entre crochets indique que, par exemple, l'élément *causer* de la valeur de $Oper_{21}$(REGRET**1**) contrôle deux ActSyntS, en plus du sujet, et donne leur réalisation, en spécifiant tous les mots-outils à introduire. Ceci est en fait le tableau de régime de cet élément de valeur de la FL. La variable *X* est la même que celle utilisée dans la description du mot-clé et renvoie à un actant de cette lexie.

Il est également possible de spécifier des conditions sur le choix de certains éléments de valeur des FL. Par exemple, toujours avec REGRET**1** (le gras est de nous):

```
{S0AntiMagn} pointe [de ~] | R. au sg
```

L'expression à la droite de la barre verticale indique que *une pointe de regrets* est agrammatical, REGRET**1** devant être au singulier pour que l'on puisse utiliser la valeur de cette fonction.

Enfin, on trouve également ce qu'on appelle des **valeurs fusionnées** des FL. Ce sont des valeurs de FL qui incluent le sens du mot-clé de la fonction. Cela s'encode en faisant précéder les valeurs fusionnées du symbole `//`. Par exemple, cette fois avec la joyeuse lexie MEURTRE (le gras est encore notre œuvre):

`{S1} auteur [de ART ~]` **`//meurtrier-n`**

Alors que *auteur* est un cooccurrent de MEURTRE, c'est-à-dire qu'il s'utilise souvent dans un énoncé avec *meurtre* (on dit *l'auteur du meurtre*), *meurtrier*, lui, s'utilise toujours sans *meurtre*; il prend en quelque sorte la place du mot-clé. En fait, les valeurs fusionnées d'une FL correspondent plutôt à des valeurs d'une autre FL. En effet, une FL est censée être ni plus ni moins qu'une étiquette apposée sur un patron régulier dans les langues. Ce patron est à la fois sémantique et syntaxique profond. Les éléments de valeur d'une même FL devraient donc tous avoir le même contenu sémantique et le même comportement syntaxique profond. Ce principe est violé avec l'utilisation des valeurs fusionnées, puisqu'alors les éléments de valeur de la FL en question ne partagent plus le même comportement syntaxique profond (ni d'ailleurs tout à fait le même contenu sémantique, puisque dans le cas des valeurs non fusionnées, le sens du mot-clé ne fait pas partie du sens des éléments de valeur, alors que le sens des valeurs fusionnées inclut toujours celui du mot-clé). C'est un problème théorique important qui met en lumière une faiblesse dans le formalisme actuel des FL. Pour une discussion sur ce sujet, voir (Kahane & Polguère, 2001).

Voilà qui met fin à notre caractérisation formelle du dictionnaire utilisé par *Sentence Garden*, le *DiCo*. Les autres champs qui apparaissent dans la fiche donnée en annexe n'encodent aucune information linguistique et ne sont destinés qu'à l'usage des humains (soit les utilisateurs, soit les lexicographes). Nous les ignorerons impitoyablement.

3.2 L'encodage du dictionnaire dans *Sentence Garden*

L'information lexicale contenue dans le *DiCo* n'est pas directement utilisable par *Sentence Garden*, puisqu'elle se trouve dans un format difficilement lisible

pour un programme Prolog. Elle doit donc être « pré-mâchée » et réécrite dans un format plus pratique. Nous présenterons plus loin une composante de *Sentence Garden* qui s'occupe justement de cela (*cf.* §3.4, p.57). Le dictionnaire utilisé par le synthétiseur est encodé sous la forme de prédicats Prolog `entry` à 8 arguments (ci-après, nous noterons les prédicats Prolog comme suit: `entry/8`) conservés dans un fichier. Chacun de ces faits correspond à la description d'une lexie. Les arguments de `entry/8` sont, dans l'ordre:

1) Le nom de la lexie.
2) Le nom du vocable auquel elle appartient.
3) Ses traits de syntactique, sous la forme d'une liste.
4) Son étiquette sémantique.
5) Sa structure actancielle (le nombre de ses actants et les étiquettes sémantiques de ces actants). Il s'agit d'une liste de couples dont le premier terme est le nom de la variable utilisée dans le *DiCo* pour référer à un actant (`'X'`, `'Y'`, …) et le second terme est l'étiquette sémantique de cet actant.
6) Une liste de ses régimes possibles. Chaque régime est lui-même une liste de triplets correspondant chacun à une ligne du tableau de régime trouvé dans le *DiCo*. Le premier terme du triplet est un nom de variable, le second un numéro d'actant syntaxique, et le dernier donne les réalisations de surface indiquées dans le *DiCo*. Le triplet dans son ensemble n'est en fait qu'un pointeur vers un patron de régime. Nous expliquerons à la prochaine section ce que nous entendons par là (*cf.* §3.3, p.53).
7) Ses fonctions lexicales. C'est une liste de quadruplets dont le premier terme est le nom d'une FL. Le second est un terme complexe composé d'un type de valeur (les types possibles sont *lexème*, *phrasème*, *discours direct* et *syntagme*), de l'opérateur « : » et finalement de l'élément de valeur de la FL. Le troisième terme correspond au régime de l'élément de valeur, et finalement le dernier correspond aux conditions. Lorsqu'il n'y a pas de régime ou de conditions dans le *DiCo* pour un élément de valeur

d'une FL, les termes correspondants dans le quadruplet sont remplis par un « ? ».

8) Quelques exemples d'utilisation de la lexie.

Dans le prototype actuel, les 3e, 4e, et 5e arguments du prédicat `entry/8` ne sont pas utilisés et sont laissés vides. À titre d'exemple, voici sous quelle forme l'article de **REGRET1** (voir l'annexe 1) est encodé dans le fichier Prolog utilisé par *Sentence Garden*:

```
entry(
  'REGRET#1',
  'REGRET#0',
  [],
  ?,
  [],
  [ [  ('X','I',[ 'de N','A-poss']),
       ('Y','II',[ '_à cause de_ N','_à propos de_ N',
                   'de V-inf']) ] ],
  [  ('Gener',lexeme:'SENTIMENT','de ~',?),
     ('QSyn',lexeme:'REPENTIR',?,?),
('QSyn',lexeme:'REMORDS',?,?),
     ('Magn•IncepFunc1',lexeme:'ASSAILLIR','N=X',?),
     ('Oper21',lexeme:'PROVOQUER','du/des ~(s) chez
       N=X',?),
     ('Oper21',lexeme:'LAISSER','des ~s à N=X',?),
     ('Oper21',lexeme:'CAUSER','ART ~ à/chez N=X',?),
     ('Magn•Oper1',phrase:'être rongé','de ~s',?),
     ('Magn•Oper1',lexeme:'SE CONSUMER','de ~',?),
     ('Oper1',lexeme:'RESSENTIR','ART ~',?),
     ('Oper1',lexeme:'ÉPROUVER',?,?),
     ('Oper1',lexeme:'AVOIR',?,?),
     ('AntiVer-utilité',lexeme:'VAIN',?,antépos),
     ('S0AntiMagn',lexeme:'POINTE','de ~','R. au sg'),
     ('AntiMagn',lexeme:'PETIT',?,antépos),
     ('AntiMagn',lexeme:'LÉGER',?,?),
     ('Magn',lexeme:'CUISANT',?,?),
     ('Magn',lexeme:'CRUEL',?,?),
     ('Magn',lexeme:'AMER',?,?),
     ('Magn',lexeme:'INFINI',?,?),
     ('Magn',lexeme:'EXTRÊME',?,?),
     ('Magn',lexeme:'VIF',?,antépos),
     ('Magn',lexeme:'PROFOND',?,?),
     ('Magn',lexeme:'GRAND',?,?),
     ('Able2',lexeme:'REGRETTABLE',?,?),
     ('MagnNonAdv1',lexeme:'SANS','l'ombre d'un ~/le
       moindre ~',?),
     ('NonAdv1',lexeme:'SANS',~,'R. sans modificateur'),
```

```
    ('Adv1',lexeme:'AVEC',~,'R. sans modificateur'),
    ('Adv1',lexeme:'À',?,?),
    ('V0',lexeme:'REGRETTER#1',?,?) ],
  'On ne savait trop si cette pointe de regret
   concernait le décès du ministre ou le caractère
   irréversible de ses décisions antérieures.' ) .
```

Le fichier que nous venons de décrire ne sert qu'à stocker l'information lexicale. Il n'est pas chargé en entier par le système. Il est parcouru lors du démarrage de *Sentence Garden* et indexé. Ce qui se trouve en mémoire, c'est alors une table donnant la position de chaque article dans le fichier de dictionnaire. Ceci évite de saturer la mémoire de l'ordinateur lorsqu'on utilise de gros dictionnaires. Grâce à cette table, le système peut trouver rapidement l'information lexicale dont il a besoin et ne charger que celle-ci. Cette information est alors transformée en objets informatiques. Les classes d'objets utilisées pour l'encodage interne du dictionnaire sont les suivantes.

Les articles de dictionnaire

Les objets de la classe `article_dict` possèdent trois attributs :

1) Une liste de traits de syntactique.

2) Une liste de fonctions lexicales (des quadruplets tels que décrits au point 7, p. 50).

3) Une liste d'objets de la classe `règle` (*cf.* § 2.2, p. 41) qui ont été créés lors du chargement de l'article de dictionnaire. Nous reviendrons sur la création automatique de ces règles lexicales à la fin du présent chapitre (*cf.* § 3.5, p. 60).

De cette classe dépendent les classes `lexie` et `vocable`. C'est ici que sont définies les méthodes qui permettent l'accès à l'information lexicale contenue dans les articles de dictionnaire.

Les lexies

Les objets de la classe `lexie`, en plus des attributs hérités de la classe `article_dict`, possèdent les cinq attributs suivants :

1) Un objet de la classe `vocable` qui encode le vocable dont la lexie en cause est une acception.
2) L'étiquette sémantique de la lexie. Cet attribut n'est pas utilisé dans le prototype actuel.
3) La structure actancielle de la lexie (telle que décrite au point 5, p. 50). Cet attribut n'est présentement pas utilisé.
4) Un pointeur vers un patron de régime (*cf.* § 3.3, p. 53).
5) Des exemples d'utilisation de la lexie.

Les vocables

La classe `vocable` est une sous-classe de `article_dict`. Ses objets, en plus des trois attributs hérités de la classe `article_dict`, ont un attribut supplémentaire. Il s'agit d'une liste d'objets de la classe `lexie` qui font partie de ce vocable.

Le dictionnaire

Enfin, la classe `dictionnaire` est une classe dont aucune instance n'est créée. Elle possède deux attributs :

1) Le chemin d'accès au fichier Prolog où est stockée l'information lexicale.
2) La liste de toutes les instances des sous-classes de `article_dict`.

Dans cette classe sont définies les méthodes de chargement et de purge de l'information lexicale, ainsi que des méthodes d'accès à l'information lexicale.

Notre présentation de l'encodage du dictionnaire dans *Sentence Garden* ne serait pas complète sans quelques mots au sujet du régime des lexies. Nous avons déjà mentionné que les tableaux de régime dans le *DiCo* étaient en fait des pointeurs vers des patrons de régime, en laissant planer le mystère. Le moment est venu de lever le voile.

3.3 Les patrons de régime

Notre travail sur *Sentence Garden* nous a révélé une faille importante dans les articles actuels du *DiCo* (et du *DEC*). Les tableaux de régime des lexies qu'on y

trouve ne sont pas assez explicites quant à la nature des relations syntaxiques de surface. En effet, ces derniers ne donnent que le nom des lexies de surface (structurelle, ou « syntaxiques ») devant se trouver dans la représentation syntaxique de surface [RSyntS] (prépositions régies, etc.) sans spécifier quelles relations relient ces lexies à leur gouverneur ou aux actants syntaxiques de la lexie qui en dépendent. Prenons par exemple le tableau de régime de la lexie REGRET1 (voir l'annexe 1):

```
X = I = de N, A-poss
Y = II = _à cause de_ N, _à propos de_ N, de V-inf
```

Alors que cette table est tout à fait explicite quant à la correspondance entre les relations sémantiques [RelSém] d'une part et syntaxiques profondes [RelSyntP] d'autre part qui lient la lexie vedette à ses actants (sémantiques et syntaxiques profonds), elle ne dit strictement rien sur la correspondance entre les RelSyntP et les relations syntaxiques de surface [RelSyntS]. Il faut donc deviner quelle RelSyntS relie la lexie décrite à la préposition DE et encore la préposition au nom réalisant le premier actant sémantique (*cf.* `X = I = de N` dans notre exemple), comme l'illustre la figure suivante.

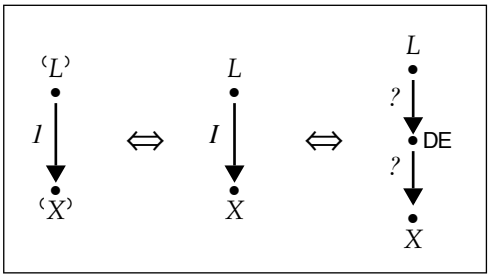


Figure 14. Une lacune dans les patrons de régime

Nous avons pallié cette situation en construisant une seconde base de données ne contenant que des patrons de régime. Chaque tableau de régime trouvé dans le *DiCo* y est répertorié[1], et pour chacun nous lui faisons correspondre un tableau

1. On trouve moins d'une centaine de tableaux de régimes différents dans la présente version du *DiCo*, pour près de 4000 lexies (merci à Adil El Ghali pour cette information). Il semble donc tout à fait possible de faire un inventaire complet de tous les patrons de régime d'une langue.

entièrement explicite quant à la nature précise des RelSyntS en jeu. Ainsi, à la table donnée plus haut nous faisons correspondre le tableau suivant :

```
X = I = {  L --Prép.--> Ω [ DE,(Prép)]
           Ω --Compl. de prép.--> X [ (N)] }
         {  L --Dét.--> X [ (A-poss)] }
Y = II = {  L --Prép.--> Ω [ _À CAUSE DE_, (Prép)]
            Ω --Compl. de prép.--> Y [ (N)] }
          {  L --Prép.--> Ω [ _À PROPOS DE_, (Prép)]
            Ω --Compl. de prép.--> Y [ (N)] }
          {  L --Prép.--> Ω [ DE, (Prép)]
            Ω --Compl. de prép.--> Y [ (V),inf] }
```

Ceci indique que, par exemple, REGRET1 —*II*→ *Y* peut se réaliser en syntaxe de surface soit par REGRET1 —*Prép.*→ DE/⌜À PROPOS DE⌝ —*Compl. de prép.*→ $Y_{[N]}$, soit encore par REGRET1 —*Prép.*→ DE —*Compl. de prép.*→ $Y_{[V],inf}$. Ce patron de régime sera valide pour toutes les lexies ayant le même tableau de régime dans leur article de *DiCo*. Ainsi, les tableaux de régimes du *DiCo* ne sont considérés que comme des pointeurs vers les patrons de régime de la base de données des régimes. Ce sont ces patrons, et non les tables du *DiCo*, qui seront utilisés par *Sentence Garden*. On peut alors se demander pourquoi ne pas « simplement » se débarrasser des tableaux actuels du *DiCo* pour y inclure directement les tableaux explicites, plutôt que d'entretenir une seconde BD. Il y a deux raisons à cela, l'une pratique, l'autre théorique.

D'un point de vue pratique, les tableaux utilisés traditionnellement dans le *DiCo* et dans le *DEC* sont d'une grande efficacité. L'humain qui écrit un article de dictionnaire, tout comme celui qui le lit, n'a habituellement pas besoin de connaître exactement la nature des RelSyntS qui relient les lexies de surface. Il est donc beaucoup plus simple d'écrire seulement

```
X = I = de N, A-poss
Y = II = _à cause de_ N, _à propos de_ N, de V-inf
```

que

```
X = I = {  L --Prép.--> Ω [ DE,(Prép)]
           Ω --Compl. de prép.--> X [ (N)] }
         {  L --Dét.--> X [ (A-poss)] }
Y = II = {  L --Prép.--> Ω [ _À CAUSE DE_, (Prép)]
            Ω --Compl. de prép.--> Y [ (N)] }
```

```
{ L --Prép.--> Ω [ _À PROPOS DE_, (Prép)]
  Ω --Compl. de prép.--> Y [ (N)] }
{ L --Prép.--> Ω [ DE, (Prép)]
  Ω --Compl. de prép.--> Y [ (V),inf] }
```

La version explicite du tableau de régime est assez peu lisible, et les risques d'erreur au moment de l'écriture sont très élevés. Considérons par exemple le régime de ABATTEMENT[2] [*Cet objectif permettra aux employeurs de bénéficier de l'abattement de 50% sur les charges sociales prévu par la loi*]:

```
X = I = de N, par N, A-poss
Y = II = de N, de Num Nom
Z = III = à N, consenti à N, pour N, A-poss
W = IV = sur N
```

Pour écrire un tableau explicite de ce schéma de régime, le lexicographe devrait spécifier la nature d'une vingtaine de RelSyntS, dans un format qui n'est pas particulièrement convivial. Quant au lecteur, la version explicite de ce tableau ne lui serait pas tellement utile, puisqu'elle lui semblerait totalement illisible. À cela s'ajoutent les difficultés qui se présenteraient à l'étape de la révision des entrées. En procédant plutôt de la façon choisie, les risques d'erreur sont considérablement réduits. Ces difficultés pourraient toutefois être contournées en utilisant un programme d'édition de dictionnaire comportant une interface graphique.

Au niveau théorique, l'entretien d'une base de données des régimes présente un certain intérêt. Cela nous permet de faire l'inventaire des patrons de régime possibles en français. Nous avons regroupé les régimes selon la partie du discours du gouverneur et le nombre de ses actants. Nous obtenons ainsi un répertoire complet (dans le domaine des lexies décrites dans le *DiCo*) des régimes nominaux, verbaux, adjectivaux ou adverbiaux à un, deux, trois, ... actants[1]. En regroupant les lexies selon leur patron de régime et en les étudiant sous cet angle, nous pourrions peut-être identifier plus facilement certains phénomènes sémantiques et syntaxiques qui nous auraient échappé autrement.

1. (Iordanskaja, 1961) propose un inventaire semblable pour le russe.

Malheureusement, nous n'avons pas pu pousser bien loin les recherches au sujet des patrons de régime du français. L'actuelle base de données des régimes dont nous disposons est très incomplète. Ce n'est toutefois qu'un début, et nous espérons avoir l'occasion d'explorer cette voie intéressante dans le futur.

Ceci complète notre présentation de la forme du *DiCo* et de son encodage dans *Sentence Garden*. Nous verrons maintenant comment il est possible de passer de façon automatique du *DiCo* au fichier Prolog mentionné plus haut (*cf.* §3.2, p.49).

3.4 L'importation et la validation du *DiCo*

Comme nous l'avons déjà dit, le *DiCo* n'est pas directement utilisable par *Sentence Garden* à cause de son format. Il doit d'abord être « digéré ». Ce travail est effectué par une composante du système, nommée *Customs*[1]. Ce programme a un double rôle : ***1)*** vérifier la validité des articles du *DiCo* et ***2)*** extraire l'information lexicale pour l'encoder dans un fichier Prolog qui pourra être utilisé par le système de synthèse (*cf.* §3.2, p.49). L'opération n'est effectuée qu'une seule fois. Par la suite, le synthétiseur ne se sert que du fichier Prolog qui a été créé, et le programme *Customs* n'a pas à être chargé en mémoire ni exécuté à chaque utilisation de *Sentence Garden*.

Le programme d'importation et de validation du dictionnaire prend en entrée un fichier au format texte exporté par *FileMaker Pro*. Chaque ligne correspond à un article de dictionnaire, et les champs sont séparés par une tabulation (c'est un format d'échange standard pour les bases de données). Le programme parcourt ce fichier et pour chaque article de dictionnaire, il en vérifie la bonne formation. Chaque champ est parcouru à l'aide d'un automate à états finis spécifique. Dans sa version actuelle, le programme vérifie la validité des tableaux de régimes et des fonctions lexicales seulement. Il permet de détecter des erreurs de syntaxe (caractères non permis, opérateurs manquants, etc.) ou de cohérence. Les articles

1. *Douanes*, en français. Le nom est une métaphore du processus effectué par ce programme : une importation de « cargos lexicaux ». Les articles qui ne répondent pas aux « normes » sont « retenus aux douanes » !

mal formés sont rejetés, alors que ceux qui sont bien formés sont réécrits dans un format Prolog.

Chaque fois que *Customs* est utilisé, il génère un rapport afin que l'utilisateur puisse voir quels articles ont été refusés, et pourquoi. L'annexe 6 montre un extrait d'un tel rapport. On voit que les types d'erreurs détectées sont assez diversifiés. Dans le cas de BARBE, le programme nous signale que le renvoi à un autre article du *DiCo* dans les valeurs des FL n'est pas géré dans la version actuelle du système. Il nous avertit également que l'article contenait une apostrophe droite, alors que lors de l'édition du *DiCo* sous la forme du *LAF* (*cf.* §3, p.44), seules les apostrophes courbées doivent être utilisées. L'erreur signalée pour CRIMINEL**1** est plus grave, car elle est de nature logique. On utilise dans l'article une variable f afin de renvoyer à une autre FL dans l'article de la lexie. Or, le programme n'a pas trouvé la définition de cette variable. Enfin, dans l'article de FARINE, on trouve une autre variable f. Celle-ci est bien définie quelque part dans l'article, mais elle manque dans la version vulgarisée du nom de la fonction, c'est-à-dire dans le nom de FL qui sera utilisé dans le *LAF*.

Enfin, en plus de générer le fichier Prolog encodant le *DiCo* et un rapport, le programme *Customs* génère également un fichier indiquant pour chaque fonction lexicale trouvée dans le *DiCo* l'équivalence entre les noms « scientifiques » et vulgarisés des fonctions lexicales. Cette information est stockée dans un fichier Prolog contenant une série de prédicats `laf/4` dont les arguments sont: ***1)*** le nom de la lexie pour laquelle la FL est définie, ***2)*** l'étiquette sémantique de la lexie (cette information est manquante dans la version actuelle du système), ***3)*** le nom vulgarisé de la FL, et ***4)*** le nom « scientifique » de la FL. Voici un extrait d'un tel fichier:

```
laf('ABANDON#I.2a',?,'Incomplet','AntiMagn') .
laf('ABANDON#I.2a',?,'Complet','Magn') .
laf('ABANDON#I.2a',?,'Résultat d'un A.','QResult') .
laf('ABANDON#I.2b',?,'Être dans un état d'A.','Oper1') .
laf('ABANDON#I.2b',?,'Indésirable','AntiBon') .
laf('ABANDON#I.2b',?,'Complet','Magn') .
laf('ABANDON#I.2b',?,'Dans un état d'A.','A1') .
```

Cette information n'est d'aucune utilité pour le synthétiseur, mais peut s'avérer très utile au lexicographe pour vérifier la cohérence des transcriptions des noms « scientifiques » des FL.

Tous ces traitements seraient impossibles sans quelques informations sur la langue dont le *DiCo* décrit le lexique. Pour chaque langue L à traiter, le programme d'importation du *DiCo* doit donc disposer de certaines informations à son sujet. Ces informations, de nature tant linguistique que métalinguistique, sont les suivantes.

Il faut d'abord la hiérarchie des étiquettes sémantiques de L. Cette hiérarchie, encodée dans un fichier Prolog, permet au programme de vérifier que la description sémantique des lexies est valide (par exemple, que les étiquettes utilisées dans le *DiCo* sont bien des étiquettes valides en L). Dans la version actuelle de *Customs*, toutefois, cette information n'est pas utilisée puisque les descriptions sémantiques ne sont pas examinées.

Ensuite, le programme d'importation du *DiCo* doit disposer d'un fichier Prolog qui contienne toute l'information reliée aux traits de syntactique possibles ou obligatoires des lexies en L. On y doit y trouver :

1) La liste des parties du discours de L. Pour chacune des parties du discours de surface existantes en L, on indique à quelle classe syntaxique majeure — c'est-à-dire à quelle partie du discours profonde — elle appartient. Par exemple, un pronom (partie du discours de surface) appartient à la classe *N(om)* (partie du discours profonde), une interjection à la classe *Adv(erbe)*, etc.
2) Les traits de syntactique qui doivent être associés aux lexies de chaque classe syntaxique majeure. C'est ici qu'on note que par exemple, en français, tout nom a un genre — *masculin* ou *féminin* — et tout verbe fait partie d'une classe conjugationnelle — *I*, *II*, *III* ou *irrégulier*.
3) La liste exhaustive des traits de syntactique qui peuvent apparaître dans la description d'une lexie. Par exemple, *pas de pl*, *performatif*, *« fam »*, etc.

Pour l'instant, comme le champ encodant les traits de syntactique des lexies n'est pas examiné par *Customs*, cette information n'est pas utilisée.

Enfin, le programme *Customs* utilise un fichier Prolog qui correspond à la base de données des patrons de régimes afin de vérifier que les tableaux de régime des articles du *DiCo* ont déjà été répertoriés. Quand un tableau de régime est bien formé mais qu'il n'est pas dans la BD des patrons de régime, il est signalé dans le rapport de *Customs*. Il revient alors au lexicographe de voir s'il n'a pas, par exemple, oublié d'inscrire une réalisation de surface possible, ou si réellement il a affaire à un régime qu'il n'avait pas encore répertorié.

Le programme *Customs*, bien qu'il ne soit encore que partiellement implémenté, a déjà permis de détecter un nombre important d'erreurs de syntaxe et de logique dans le *DiCo*. Il ne fait aucun doute que ce type de programme est un outil précieux pour le lexicographe.

3.5 L'encodage du dictionnaire sous forme de règles

Nous avons vu jusqu'ici comment la grammaire et le dictionnaire sont encodés dans *Sentence Garden*. Dans la plupart des systèmes de traitement automatique des langues, le dictionnaire est un ensemble plus ou moins structuré d'informations linguistiques auxquelles font appel les règles de la grammaire. À notre connaissance, tous les systèmes basés sur la TST réalisés jusqu'ici suivent ce modèle. Pourtant, dans le cadre de cette théorie, les dictionnaires sont conçus comme des ensembles de règles lexicales. Les dictionnaires tels que nous les construisons, de type *DEC* ou *DiCo*, ne sont rien d'autre qu'un « raccourci » formel; en fait ils devraient idéalement être encodés selon exactement le même formalisme que les grammaires.

Nous avons voulu tester cette idée avec *Sentence Garden*, ne serait-ce que partiellement. Ainsi, lors du chargement d'un article de dictionnaire, le système crée automatiquement des règles lexicales, encodées selon le même formalisme que les règles grammaticales, qu'il ajoute aux modules pertinents (dans notre cas,

uniquement le module sémantique). Dans le prototype actuel, les règles lexicales créées automatiquement sont ***1)*** une règle de transition établissant la correspondance entre le sémantème exprimé par la lexie et la lexie elle-même et ***2)*** des règles d'équivalence sémantique indiquant la correspondance entre le sémantème qu'exprime la lexie et celui de ses (quasi-)synonymes ainsi que ses équivalents verbaux, nominaux, etc. Ces règles d'équivalence encodent l'information donnée par les FL *Syn*, *QSyn*, V_0, S_0, A_0 et Adv_0, dans le même langage formel que les règles grammaticales présentées plus haut (*cf.* §2.1, p.39). Dans une version ultérieure de *Sentence Garden*, il serait assez facile d'ajouter à cela la création automatique de règles de régime en utilisant les patrons de régime explicites que nous proposons (*cf.* §3.3, p.53), puisqu'ils sont équivalents à des règles.

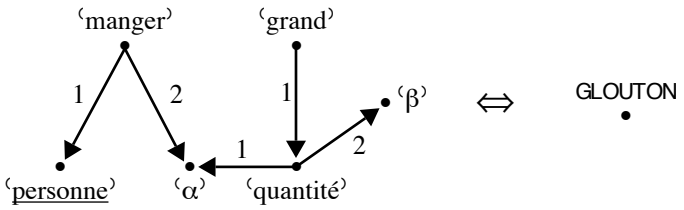
Le principal avantage d'encoder le dictionnaire sous forme de règles est que le moteur de synthèse n'a plus qu'un seul type d'objet à manipuler: toutes les règles linguistiques, qu'elles soient lexicales ou grammaticales, sont écrites dans le même formalisme. Il n'est alors plus nécessaire de prévoir des procédures d'accès à l'information lexicale dans des dictionnaires encodés de façon plus ou moins formelle.

Un autre avantage est la portabilité que cela confère au dictionnaire. Il existe en ce moment plusieurs outils informatiques manipulant des MST. Tous utilisent à peu près le même type de règles grammaticales, mais chacun utilise un encodage différent pour l'information lexicale. En encodant nos dictionnaires selon le même formalisme que nos grammaires, nous pourrions beaucoup plus facilement échanger d'un système à l'autre les informations linguistiques qu'ils contiennent.

Enfin, un dictionnaire encodé sous forme de règles « standard » est moins redondant que les dictionnaires du type *DEC*. En effet, une même règle peut encoder des informations pertinentes à plusieurs lexies à la fois. Par exemple, la

règle (approximative) suivante équivaut autant à la définition de GLOUTON qu'à la FL $Magn_2^{quant}+S_1–personne($MANGER$)$ = GLOUTON.

Règle 1. Le sens de GLOUTON



Il va sans dire que d'encoder tout un dictionnaire selon le même formalisme que les règles grammaticales et inclure ces règles dans les modules linguistiques aurait pour effet d'augmenter considérablement le nombre de règles que le moteur de synthèse d'un système doit manipuler. Il devient alors crucial d'avoir un algorithme de matching et d'application des règles très efficace. Nous verrons au chapitre suivant que de ce point de vue, notre prototype de *Sentence Garden* n'est malheureusement pas équipé pour gérer un tel nombre de règles.

Chapitre IV. La synthèse dans *Sentence Garden*

Le présent chapitre vise à expliquer d'un point de vue procédural comment s'effectuent les transitions qui composent la synthèse linguistique (*cf.* ch.I, §4, p.17), et comment le moteur de synthèse (*cf.* ch.II, §1, p.23) gère la production de paraphrases.

1 L'algorithme général de synthèse dans *Sentence Garden*

Nous avons vu au chapitre I que la synthèse pouvait être considérée comme une série de microsynthèses (*cf.* ch.I, §4, p.17). C'est exactement de cette façon qu'elle s'effectue dans *Sentence Garden*. On peut représenter la procédure en pseudo-code de la façon suivante en supposant ***1)*** une file[1] contenant au départ une RSém et ***2)*** une fonction `microsynthèse(`*X*`)` qui construit et retourne comme valeur une file contenant tous les produits de *X* (*cf.* §5, p.77):

```
retirer X de la file
Produits = microsynthèse(X)
ajouter chaque élément de Produits à la file
si ( la file n'est pas vide
     et
     le prochain élément de la file n'est pas un texte )
  alors répéter
```

Ce qui reste dans la file à la fin de ce processus est l'ensemble des textes exprimant la RSém de départ (cet ensemble pouvant être vide si les règles de la grammaire sont insuffisantes pour réaliser cette RSém ou si cette dernière était mal formée).

---

1. Cette structure de données est mieux connue sous le nom anglais de *first in first out*, puisque le premier élément à être mis en file sera le premier à en sortir. Les files s'opposent en quelque sorte aux piles, aussi connues sous leur nom anglais de *last in first out*, pour lesquelles le premier élément entré ressort le dernier.

Ce qui nous intéresse ici, c'est comment s'effectuent ces microsynthèses; nous proposons de l'étudier dans les sections qui suivent. Notre prototype de *Sentence Garden* n'effectue que la première microsynthèse (ce que plusieurs appellent *l'interface sémantique-syntaxe*), mais l'approche devrait être sensiblement la même pour toutes les microsynthèses; le système peut donc être modifié pour effectuer une synthèse complète. Les étapes qui ne sont propres qu'à la microsynthèse RSém ⇒ RSyntP seront explicitement identifiées comme telles.

La première étape d'une microsynthèse est d'établir un ordre de parcours de la représentation source. Puis, les éléments de la représentation source sont réalisés, un à un ou en petits groupes, selon les règles disponibles dans le MST.

2 Le parcours de la représentation source

2.1 Problématique

Les règles d'un MST ne s'appliquant jamais à des représentations entières mais plutôt à des éléments de représentations, on ne peut pas réaliser une représentation d'un seul coup: il faut procéder par petits blocs. Or, un problème se pose: quels sont les éléments qui doivent être réalisés en premier? Y a-t-il un ordre à respecter? Lorsque la source est une RMorph — dont la structure porteuse est une chaîne — le problème peut être résolu de façon simple et intuitive: on commence par le premier nœud de la chaîne, puis on passe au second, et ainsi de suite. Par contre, dès qu'on a affaire à une RSynt — dont la structure porteuse est un arbre — le choix est moins évident. S'il semble que l'on doit commencer par le sommet, il est plus difficile de savoir s'il est préférable de suivre un parcours en profondeur d'abord puis en largeur, ou au contraire de parcourir l'arbre d'abord en largeur

avant de descendre en profondeur. La figure suivante illustre ces deux cas ; les numéros indiquent l'ordre de parcours des nœuds[1].

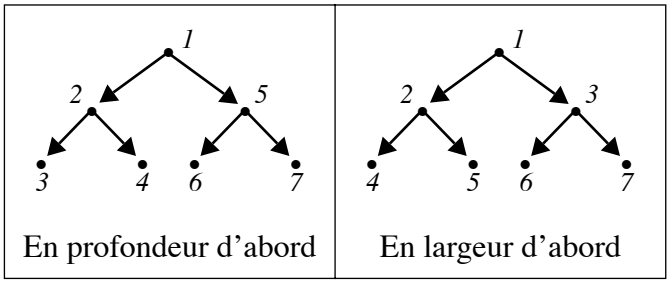


Figure 15. Deux types de parcours des arbres syntaxiques

Enfin, lorsque la source est une RSém — dont la structure porteuse est un réseau — l'ordre dans lequel les éléments doivent être réalisés est loin d'être évident. D'abord, alors que les SSynt et SMorph possèdent un nœud en quelque sorte « naturellement dominant » (le sommet de l'arbre pour les SSynt et le premier nœud de la chaîne pour les SMorph), aucun élément d'une SSém n'apparaît d'emblée comme dominant. Ensuite, une fois qu'on a décidé par quel nœud débuter, le nombre de parcours possibles à partir de ce nœud augmente avec la complexité du réseau.

Le problème peut sembler peu ou pas du tout pertinent. D'ailleurs, dans la plupart des synthétiseurs de type Sens-Texte développés jusqu'ici[2], il est totalement ignoré. La méthode classique consiste à procéder de façon plus ou moins aléatoire : on réalise chaque élément de la source, dans le désordre, créant un certain nombre d'éléments dans les représentations produits qu'il ne nous reste plus qu'à « coller » ensemble. Cette technique est expliquée notamment dans (Boyer & Lapalme, 1985). Or, elle pose certaines difficultés, surtout pour la microsynthèse RSém ⇒ RSyntP, les deux principales étant la construction de la SSynt-AnaphP et la construction de relations ATTR entre deux nœuds syntaxiques profonds. Ces problèmes sont plus facilement résolus en adoptant la technique

1. Il existe d'autres types de parcours pour les arbres. Par exemple, on peut imaginer un parcours ascendant, c'est-à-dire commençant par les feuilles et remontant vers le sommet.
2. *Gossip* (Iordanskaja *et al.*, 1988) et *LFS* (Iordanskaja *et al.*, 1992) sont des exceptions à la règle.

proposée ici. Nous verrons également que cette idée de réaliser les éléments de la source dans un ordre précis correspond à autre chose qu'un simple ordonnancement.

On constate que le problème du parcours de la source peut être divisé en deux problèmes plus simples: ***1)*** choisir un nœud par lequel commencer, ce que nous appellerons le **nœud d'entrée**, et ***2)*** établir un ordre de parcours des autres nœuds à partir du nœud d'entrée, ce que nous appellerons l'**exploration** de la source.

Nous laisserons de côté la question du parcours des RSynt et des RMorph pour nous concentrer sur le cas des RSém, puisque c'est à ce niveau de représentation que le problème est le plus complexe et le plus pertinent. Dans le cas d'une RSém, contrairement aux autres types de représentations, la structure porteuse (la SSém) à elle seule n'est pas hiérarchisée et n'impose pas un « parcours évident ». Nous utilisons donc la SSém-Comm pour guider notre parcours de la SSém. Ainsi, la notion de parcours est intimement liée à l'établissement d'une **hiérarchie communicative**[1] entre les nœuds d'une représentation. Il est fort possible qu'il soit utile, sinon nécessaire, de faire appel à la SSynt-CommP pour établir le parcours d'une SSynt lors des microsynthèses RSyntP $\Rightarrow$ RSyntS ou RSyntS $\Rightarrow$ RMorphP (par exemple, parcourir tout le rhème avant de parcourir le thème — ou vice-versa). Toutefois, à notre connaissance, aucune recherche n'a été faite dans ce sens jusqu'ici.

Les règles de parcours des représentations ne font pas partie d'un MST d'une langue: elles sont universelles, puisqu'elles ne dépendent que de la forme des représentations. Pour cette raison, elles ont été directement intégrées à *Sentence Garden*. Elles correspondent vraisemblablement à une réalité psychique (liée à la physiologie humaine), à la façon dont les humains manipulent l'information

1. Expliquer en détail la notion de hiérarchie communicative demanderait beaucoup trop d'espace; nous ne pouvons qu'effleurer le sujet en simplifiant au maximum. Le lecteur curieux pourra consulter (Polguère, 1990) ainsi que (Mel'čuk, 2001) pour une présentation plus développée de cette notion.

linguistique et produisent des énoncés. Lorsqu'un locuteur parle, certains éléments sont plus saillants à son esprit, et c'est tout comme s'il construisait sa phrase à partir de ces éléments, plus ou moins en accrochant des bouts d'information à ce noyau. Évidemment, ce n'est que pure spéculation. La question dépasse largement le cadre de ce mémoire, mais l'universalité, la simplicité et l'efficacité des règles de hiérarchisation communicative portent à croire qu'elles correspondent à un processus mental réel.

2.2 Le choix du nœud d'entrée

Le nœud d'entrée d'une RSém est *grosso modo* le nœud le plus saillant communicativement. C'est ce nœud qui sera réalisé comme le sommet des RSyntP correspondantes[1]. Il existe quelques règles procédurales relativement simples pour choisir le bon nœud d'entrée. Ces règles sont introduites dans (Iordanskaja & Polguère, 1988; Iordanskaja, 1990; Polguère, 1990; Mel'čuk, 2001: 38-48). La procédure utilisée dans *Sentence Garden* est une version simplifiée de celle décrite dans (Polguère, 1990). Toutefois, plutôt que de choisir un seul nœud d'entrée, notre système utilise l'un après l'autre tous les nœuds d'entrée possibles, de façon à obtenir un plus grand nombre de paraphrases. Les nœuds d'une RSém pouvant servir d'entrée sont (dans cet ordre de préférence):

1) Le nœud dominant du rhème.

2) Le nœud dominant du thème.

3) N'importe quel nœud qui se trouve sur le chemin entre le nœud dominant du rhème et celui du thème mais qui ne fait partie d'aucun sous-réseau communicatif[2]. Dans le reste de l'exposé, nous ignorerons ce troisième cas.

---

1. Parfois, un verbe support doit être utilisé. Alors, c'est ce verbe support qui est le sommet et le nœud SyntP correspondant au nœud d'entrée en est un dépendant immédiat.
2. Voir la notion de chemin communicatif dans (Polguère, 1990).

Par exemple, supposons la simple RSém (partielle) suivante :

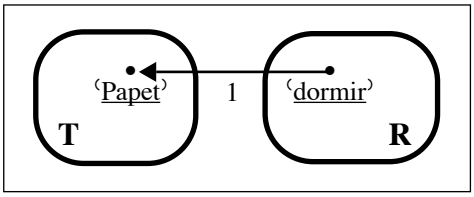


Figure 16. Deux nœuds d'entrée potentiels

Chacun des deux nœuds de cette RSém est un nœud d'entrée potentiel. À chaque nœud d'entrée correspond une famille de paraphrases, tel qu'illustré dans la figure suivante :

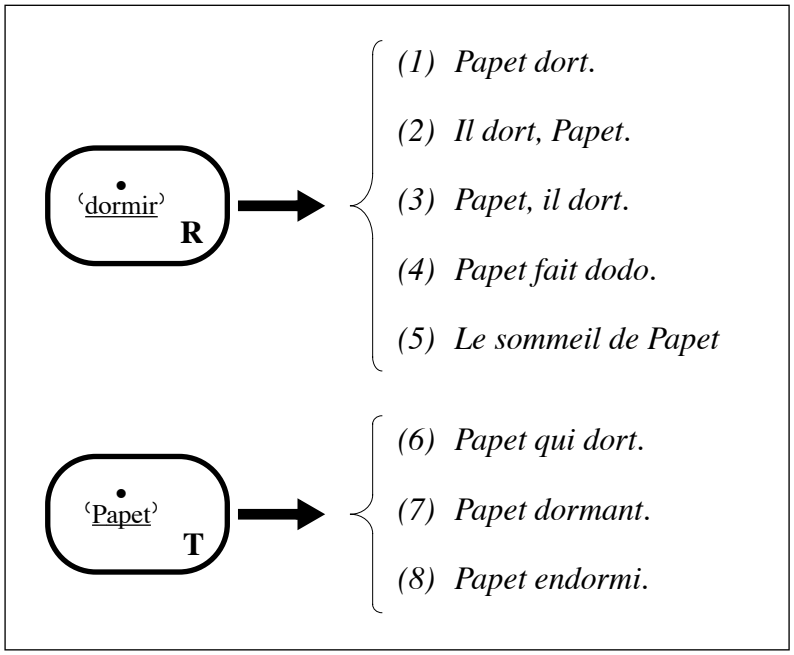


Figure 17. Deux familles de paraphrases

Bien entendu, les phrases (5) à (8) prises hors contexte peuvent être surprenantes. Ce sont des phrases « dégénérées », puisque ce ne sont que des groupes nominaux. Il existe néanmoins pour chacune un contexte où elles sont tout à fait appropriées. Par exemple, dans une comédie, la phrase (6) pourrait être ajoutée en aparté après qu'un personnage ait présenté Papet.

Rem: Il n'est pas clair que pour les phrases (6) à (8) la SSém-Comm indiquée ici soit la bonne. En effet, il est possible qu'un groupe nominal ne puisse pas contenir à la fois un thème et un rhème, et qu'en fait les phrases (6) à (8) n'étant que des thèmes ne sont pas des phrases (*cf.* Mel'čuk, 2001:103).

> Nous laissons de côté cette question épineuse. Nous voulons que *Sentence Garden* soit capable d'accepter en entrée des RSém marginales ou incomplètes (du moment qu'elles soient bien formées) et synthétise tout ce qu'il peut à partir de ces RSém. Si l'utilisateur donne en entrée la RSém en figure 16, le système doit en fournir toutes les réalisations possibles, même celles qui sont douteuses hors contexte. Éventuellement, le système devrait signaler les phrases douteuses, mais pour l'instant notre prototype ne fait pas cela.

Dans la plupart des systèmes existants, le sommet de toute RSyntP doit être verbal. Le nœud d'entrée est donc choisi en fonction de contraintes provenant d'un niveau adjacent et non de contraintes liées à la RSém elle-même. Nous n'avons pas voulu que *Sentence Garden* procède ainsi, parce que ***1)*** dans certaines langues il est tout à fait banal qu'une phrase ait un sommet syntaxique non verbal, et nous voulions que *Sentence Garden* soit capable de traiter des MST de n'importe quelle langue, et ***2)*** on peut souhaiter synthétiser des titres, qui, même en français et dans les autres langues européennes, ont souvent un sommet syntaxique nominal ou même adverbial.

2.3 L'exploration de la représentation source

Une fois que le nœud d'entrée a été choisi, on explore les arcs qui lui sont attachés pour « visiter » ses nœuds filles et ses nœuds mères, puis on explore encore les arcs qui sont attachés à ces filles et ces mères, et ainsi de suite. Il y a toutefois un certain ordre à respecter. La procédure est définie comme suit pour tout nœud N:

1) On suit récursivement les chemins descendants qui partent de N.

2) On remonte aux nœuds mères de N, et on reprend l'exploration à partir de chacune de ces mères (c'est-à-dire qu'on suit les étapes ***1)*** à ***3)*** à nouveau, mais cette fois à partir de chacun de ces nœuds).

3) On reprend l'exploration à partir de chacune des mères de tous les dépendants (directs ou indirects) de N.

Toute exploration doit respecter les contraintes suivantes:

1) Un arc n'est jamais parcouru plus d'une fois.

2) Lorsqu'on parcourt de façon descendante un arc qui relie un nœud N_i du sous-réseau communicatif SR_i à un nœud N_j du sous-réseau communicatif SR_j et qu'au moins un de ces nœuds n'est pas le nœud dominant du sous-réseau auquel il appartient, on ne poursuit pas l'exploration de SR_j au-delà de N_j.

3) On ne parcourt jamais un arc de façon ascendante si cet arc relie deux nœuds qui n'appartiennent pas au même sous-réseau communicatif et que le nœud fille ne domine pas communicativement sa mère.

L'ordre dans lequel les nœuds sont visités en suivant cette procédure correspond à la hiérarchie communicative entre ces nœuds: un nœud N_1 domine communicativement un nœud N_2 si N_1 a été visité avant N_2.

Lorsqu'un nœud est visité deux fois (si, par exemple, il a deux nœuds mères), il donnera lieu à la création de deux nœuds distincts reliés par une relation anaphorique dans la représentation cible.

Une fois que le moteur de synthèse a établi tous les parcours possibles, il crée autant de représentations produits qu'il y a de parcours. Ces produits sont vides au départ, mais seront éventuellement remplis. Le système doit garder la trace du parcours qui est associé à chacun de ces produits.

2.4 La notion de parcours dans un synthétiseur de paraphrases

La notion de parcours des représentations sources est très importante pour un système comme *Sentence Garden*. La réalisation des éléments d'une source se fait en suivant les différents ordres de parcours possibles. À chaque parcours correspond une famille de paraphrases (deux phrases synthétisées à partir d'un même parcours étant des paraphrases plus exactes que des phrases issues de parcours distincts). En ce sens, *Sentence Garden* est un système qui utilise principalement la structure communicative pour gérer la production de paraphrases. Notons qu'il existe tout de même d'autres sources de paraphrases,

notamment le choix lexical (Polguère, 2000c ; Wanner, 1996), qui n'est pas vraiment utilisé dans notre système.

3 La procédure de transition

Nous savons qu'une microsynthèse est la résultante d'un certain nombre de transitions concurrentes (*cf.* ch. I, § 4, p. 17). Cela entraîne qu'un synthétiseur de paraphrases doit, lors d'une microsynthèse, gérer la construction en parallèle de plusieurs produits. Il ne peut pourtant travailler que sur un seul produit à la fois. Le moteur de synthèse doit donc fonctionner comme un horloger qui serait en train d'assembler plusieurs montres en même temps. L'artisan a devant lui, posées sur sa table, toutes les montres en chantier. Pour chacune il se rappelle où il est rendu dans le processus d'assemblage. Pourtant, il ne peut porter son attention que sur une seule montre à la fois. Il les prend donc à tour de rôle : il ajoute une pièce à une première montre, la remet en place, puis ajoute une pièce à une seconde montre, qu'il remet ensuite sur sa table de travail. C'est exactement comme ça que procède le synthétiseur. Bien qu'il y ait plusieurs représentations produits en chantier, le moteur de synthèse ne travaille jamais que sur un seul produit à la fois. Le produit sur lequel le synthétiseur « porte son attention » à un moment précis s'appelle la **cible**.

La transition des éléments d'une représentation source se fait en suivant l'ordre de parcours établi (en fonction du parcours de la source qui est associé à la cible). Le système commence par réaliser le nœud d'entrée, puis le premier arc du parcours, ensuite le nœud au bout de cet arc, et ainsi de suite jusqu'à ce que tous les éléments de la source aient été réalisés. Pour chaque élément, le moteur de synthèse cherche d'abord toutes les règles qui peuvent être matchées sur cet élément (c'est-à-dire les règles qui permettent de réaliser ce dernier), puis applique chacune de ces règles avant de passer au prochain élément dans le parcours.

4 La procédure de matching des règles

Nous appelons **matching** l'opération qui consiste à vérifier que les conditions d'application d'une règle sont remplies dans un contexte donné, c'est-à-dire en tenant compte d'une source et d'une cible précises.

Rem: Nous avons choisi d'utiliser un anglicisme d'abord parce qu'il n'existe en français aucun terme qui nous convienne parfaitement. Celui qui se rapproche le plus de l'idée que nous voulons exprimer est *appariement*. Or, il est déjà utilisé dans (Milićević, à paraître) et (Milićević, 1997) avec un sens très différent de celui que nous aimerions lui donner. Enfin, le terme *matching* (ainsi que le verbe *matcher*) est déjà assez couramment utilisé par des francophones pour exprimer justement l'idée que nous cherchons à exprimer.

Une règle est applicable si les trois conditions suivantes sont simultanément remplies:

1) Sa partie gauche matche avec la source.

2) Sa partie droite matche avec la cible.

3) La valeur de vérité de la formule logique exprimant ses conditions d'application est *VRAI*.

4.1 Le matching des parties gauche et droite des règles

Le matching des parties gauche et droite d'une règle se fait *grosso modo* de la même façon, la seule différence étant que les éléments actifs (*cf.* ch. III, § 2.1, p. 39) de la partie droite de la règle n'ont pas à être matchés, puisqu'ils désignent des éléments qui ne seront créés que si la règle est appliquée, donc ils n'ont forcément pas de nœud correspondant dans la cible. Pour tous les autres éléments (éléments actifs à gauche et contextuels à gauche et à droite), on matche d'abord les nœuds, puis les arcs.

D'abord, pour chaque nœud N_ρ de la règle à matcher, on vérifie qu'il existe dans la représentation en question (soit la source ou la cible) un nœud N_R tel que toutes les conditions suivantes sont vérifiées:

1) N_R n'a pas déjà été matché à un autre nœud $N_\rho{}'$ de la même règle.

2) Si N_ρ est actif, N_R doit être disponible, c'est-à-dire que son correspondant n'a pas encore été construit dans la cible.

3) L'étiquette de N_ρ peut s'unifier avec celle de N_R.

4) Chaque élément qui est attaché en indice à N_ρ peut être unifié avec un élément attaché en indice à N_R.

Nous entendons par **unification** l'opération suivante (qui correspond à l'unification en Prolog). X et Y peuvent être unifiés si et seulement si une des conditions suivantes est vérifiée:

1) X et Y sont identiques.

2) Un des deux termes est une variable. Elle prendra alors la valeur de l'autre terme.

3) X et Y sont tous les deux des variables; elles seront alors liées, c'est-à-dire qu'elles devront s'instancier par la suite à la même valeur.

Ensuite, pour chaque arc A_ρ reliant le nœud ${N_\rho}^{\text{gouv}}$ à son dépendant ${N_\rho}^{\text{dép}}$ de la règle à matcher, on vérifie qu'il existe dans la représentation en question (soit la source ou la cible) un arc A_R reliant le nœud ${N_R}^{\text{gouv}}$ à son dépendant ${N_R}^{\text{dép}}$ tel que toutes les conditions suivantes sont vérifiées:

1) A_R n'a pas déjà été matché à un autre arc $A_\rho{}'$ de la même règle.

2) ${N_\rho}^{\text{gouv}}$ a été matché à ${N_R}^{\text{gouv}}$ et ${N_\rho}^{\text{dép}}$ à ${N_R}^{\text{dép}}$.

3) L'étiquette de A_ρ peut s'unifier avec celle de A_R.

4) L'orientation communicative entre ${N_\rho}^{\text{gouv}}$ et ${N_\rho}^{\text{dép}}$ (indiquée par le soulignement du nœud communicativement dominant dans la règle) est la même qu'entre ${N_R}^{\text{gouv}}$ et ${N_R}^{\text{dép}}$. Cette orientation est déterminée par le parcours de la source (*cf.* §2, p.64).

4.2 La vérification des conditions d'application des règles

Les conditions d'application des règles s'encodent en utilisant la logique des prédicats (*cf.* ch.III, §2.1, p.39). Cela en facilite beaucoup le traitement, particulièrement en Prolog. Toutefois, la formule exprimant les conditions

d'application d'une règle peut contenir un mélange de prédicats logiques et de fonctions. *Sentence Garden* inclut donc un interpréteur qui définit la sémantique des prédicats et des fonctions qui peuvent être utilisés dans les conditions des règles. Les prédicats qui sont reconnus par cet interpréteur sont les suivants :

- `=(`N`,`SG`)` sera évalué à *VRAI* ssi le nœud N fait partie du sous-graphe communicatif SG.
- `=(`X`,`Y`)` sera évalué à *VRAI* ssi X et Y peuvent s'unifier.
- `mother(`N_1`,`N_2`)` sera évalué à *VRAI* ssi N_1 est matché à un nœud dont dépend directement le nœud auquel N_2 est matché.
- Tous les opérateurs logiques peuvent également être utilisés et conservent leur sémantisme.

À cela s'ajoute la fonction `lex(`S`)`, qui retourne pour le sens S son lexème correspondant. Également, toutes les fonctions lexicales peuvent être utilisées. La valeur retournée par la fonction est alors une lexie. Cette information est extraite directement du dictionnaire. Donc, par exemple, la condition `magn(`L_1`)` = L_2 sera évaluée à VRAI ssi le dictionnaire indique que la lexie L_2 est un élément de valeur possible de la fonction *Magn* appliquée à la lexie L_1.

4.3 Les problèmes liés à la procédure de matching des règles

Cette procédure de matching des règles est simple, mais loin d'être optimale. D'une part, le matching des nœuds et celui des arcs se font de manière indépendante, l'un après l'autre, et dans cet ordre. Ainsi, les arcs attachés à un nœud ne sont jamais pris en compte lors du matching de ce nœud. Ce n'est donc que lorsqu'on tentera de matcher les arcs qu'on se rendra compte que le matching des nœuds était incorrect, et on devra alors recommencer toute l'opération à partir du début. Dès qu'on a affaire à des représentations un peu lourdes ou des règles assez complexes, où apparaissent plusieurs nœuds, on risque de devoir refaire les mêmes opérations plusieurs fois avant d'arriver à un matching des nœuds qui permette de matcher également les arcs. Le système *MATE* procède différemment.

Dans ce système, le matching des nœuds et celui des arcs ne sont pas effectués en deux étapes distinctes. Le système choisit d'abord un nœud (n'importe lequel) de la règle à matcher, puis tente de le matcher ***en tenant compte des arcs qui y sont attachés***. Il tente ensuite de matcher ces arcs, puis les nœuds qui se trouvent au bout de ces arcs, et ainsi de suite. Cet algorithme semble être plus efficace.

D'autre part, lors du matching des règles, le synthétiseur ne prend en considération que la représentation cible, en ignorant complètement tous les autres produits concurrents, qui sont totalement indépendants. Or, des produits concurrents sont habituellement très similaires. Ainsi, une règle qui peut s'appliquer dans le contexte d'une source et d'une cible données risque fort de pouvoir également être appliquée dans le contexte de la même source et d'autres produits concurrents. On se trouve alors à répéter inutilement les mêmes opérations de matching. Qui plus est, toutes les opérations subséquentes d'application des règles seront également dupliquées. (Nasr, 1996: 128-134) montre clairement que le genre de solution adoptée dans *Sentence Garden* est inefficace et propose une solution basée sur les piles à structure de graphe (Tomita, 1988); elle consiste *grosso modo* à factoriser ce qui est commun à plusieurs produits de façon à éviter la duplication des opérations.

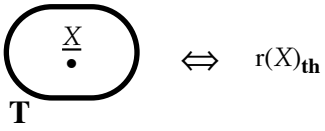
Par ailleurs, l'algorithme de matching des règles de *Sentence Garden* entraîne un autre problème lié à la performance. Dans la version actuelle du système, chaque règle du MST doit être examinée, une à une, pour voir si elle peut être matchée. Cette façon de procéder n'est viable que pour un prototype qui ne compte que peu de règles; dans un système comprenant beaucoup de règles, il faut absolument utiliser un algorithme plus efficace. Il serait intéressant d'indexer les règles de façon à ce que seules les règles susceptibles d'être applicables soient examinées, ou encore qu'on puisse matcher ensemble des blocs de règles apparentées. (Bohnet, 1999) explore cette idée.

Dans le cadre de *Sentence Garden*, nous n'avons pas cherché à optimiser le matching des règles parce que nous croyons que c'est une question beaucoup plus

complexe qu'elle n'y paraît, et les contraintes de temps ne nous permettaient pas de l'aborder.

Enfin, un problème de nature différente se pose dans le cas des règles dont les éléments actifs à gauche ne sont ni des nœuds, ni des arcs. Par exemple, le japonais a un suffixe, -WA, qui marque le thème de la phrase. Il existe donc une règle syntaxique de surface qui ressemble à ceci[1] :

Règle 2. Une règle SyntS du japonais



Cette règle établit une correspondance entre une configuration communicative et un grammème, sans qu'il n'y ait aucun nœud ni aucun arc actif à gauche. Or, la procédure de matching des règles, dans la version actuelle du système, ne permet pas de traiter ce genre de règles. C'est une lacune que nous n'avons pas tenté de combler : *Sentence Garden* n'étant qu'un premier prototype, il a fallu laisser de côté plusieurs problèmes pourtant très pertinents comme celui-ci.

4.4 Vers un matching souple des règles linguistiques

Outre ces problèmes qu'il faut corriger, nous croyons qu'il serait utile d'utiliser un algorithme de matching souple qui permettrait de matcher de façon approximative les règles et les représentations. Nous avons vu plus haut que le matching d'une règle consistait à vérifier les trois conditions suivantes :

1) Sa partie gauche matche avec la source.

2) Sa partie droite matche avec la cible.

1. Cette règle est incomplète. Elle devrait notamment tenir compte de la saillance communicative du thème par rapport au sommet syntaxique. En effet, le thème ne prend aucune marque quand il est en arrière-plan (et il est alors placé après le sommet). Ainsi le suffixe - WA est présent dans *korewa oisii* ‹ça, c'est bon›, mais pas dans *oisii, kore* ‹c'est bon, ça›.

3) La valeur de vérité de la formule logique exprimant ses conditions d'application est *VRAI*.

Dans *Sentence Garden* comme dans les autres systèmes de génération ou de synthèse que nous connaissons, l'évaluation de ces conditions se fait en utilisant une logique classique à deux valeurs de vérité : 0 (*FAUX*) et 1 (*VRAI*). Ainsi, soit toutes les conditions sont pleinement vérifiées (alors la règle s'applique), soit elles ne le sont pas (alors la règle ne s'applique pas). Le matching souple que nous proposons serait plutôt basé sur la logique floue (Zadeh, 1965), dans laquelle il existe un nombre infini de valeurs de vérité : une règle pourrait donc matcher « plus ou moins » avec une source et une cible. Selon son « degré de matching », qui serait calculé au moyen de règles de quasi-équivalence[1], l'application de la règle en question mènerait à la construction de produits plus ou moins fidèles à la source. On obtiendrait ainsi non seulement toutes les phrases exprimant exactement le sens de départ, mais également toutes celles qui l'expriment à peu près. Nous croyons qu'un tel système modéliserait de façon plus réaliste la manière dont les humains manipulent les sens linguistiques et pourrait être fort utile, notamment dans les applications de traduction automatique ou d'aide à la rédaction.

5 La procédure d'application des règles

Une fois qu'on a trouvé toutes les règles qui matchent avec un élément d'une représentation, il reste encore à les appliquer. Cette procédure est relativement simple. Nous pouvons l'illustrer de la façon suivante. Le système dispose de ***1)*** la cible actuelle, ***2)*** une file contenant toutes les règles concurrentes qui ont été matchées pour l'élément que l'on est en train de réaliser (à chacune de ces règles est associée une table de matching indiquant à quels éléments de la source et de la cible ont été matchés les éléments des parties gauche et droite de la règle), ***3)*** une fonction `dupliquer(`*X*`)` qui retourne un clone de la représentation *X*, ***4)*** une procédure `appliquer(`*X*`,`*Y*`)` qui ajoute les éléments actifs de la partie droite de la

1. Nous ne sommes malheureusement pas encore en mesure de donner un exemple d'une telle règle ; c'est une recherche qui dépasse largement le cadre de ce mémoire.

règle *X* à la représentation *Y*, et ***5)*** une file de produits, vide au départ, dans laquelle seront stockés tous les produits.

```
retirer Règle de la file de règles
si la file de règles n'est pas vide
  alors
    Clone=dupliquer(Cible)
    appliquer(Règle,Clone)
    ajouter Clone à la file de produits
    répéter
  sinon
    appliquer(Règle,Cible)
    ajouter Cible à la file de produits
```

La file de produits qui reste à la fin correspond à la valeur retournée par la fonction `microsynthèse(X)` dans la procédure générale décrite plus haut (*cf.* § 1, p. 63).

La procédure `appliquer(X,Y)` ne fait que créer de nouveaux nœuds, arcs, etc., selon les spécifications de la règle *X*. Les éléments à créer sont les éléments actifs de la partie droite de la règle. Une fois ces éléments créés, ils sont ajoutés à la cible *Y* et reliés aux éléments déjà existants dans cette représentation.

Voilà qui termine notre présentation de la procédure de synthèse dans *Sentence Garden*. Nous allons maintenant conclure en revenant sur les points les plus importants de ce mémoire.

Chapitre V. Conclusion

Nous avons présenté un système de synthèse automatique de paraphrases, *Sentence Garden*. L'originalité de ce système réside principalement dans le fait qu'il vise à produire non pas une seule phrase, mais toutes les phrases possibles qui expriment la même RSém. Ainsi, il modélise la langue, et non pas un locuteur. Nous avons également proposé un type de matching souple, basé sur la logique floue, qui permettrait de produire encore plus de paraphrases à partir d'une même RSém.

Ce système de synthèse est destiné à être utilisé par des linguistes afin de vérifier leurs modèles linguistiques de façon expérimentale, conférant ainsi une valeur scientifique à ces modèles. Il a déjà permis de repérer de nombreuses erreurs dans le *DiCo*, et il est clair qu'il peut être très utile pour le travail des lexicographes, notamment grâce à son module de validation de dictionnaires, *Customs*. Le synthétiseur pourrait également être utilisé comme composante d'un système plus complexe, par exemple un générateur de textes, ou encore un système de traduction automatique, en tant que « serveur linguistique ».

Notre travail sur la synthèse de textes a révélé une faille dans les tableaux de régime du *DEC* et du *DiCo*. En effet, les relations syntaxiques de surface entre les lexies et leurs actants ne sont pas données par ces tableaux. Nous avons donc proposé d'entretenir une base de données des patrons de régimes pour chaque langue.

Nous proposons également d'encoder le dictionnaire en utilisant le même formalisme que celui que nous utilisons pour la grammaire : des règles de transition et des règles d'équivalence. Nous avons fait quelques expériences dans le cadre de *Sentence Garden* de transformation automatique du *DiCo* en règles de ce type. Le

principal avantage d'encoder le dictionnaire selon le même formalisme que la grammaire est que les deux composantes principales du MST se trouvent alors décrites dans le même format, ce qui facilite beaucoup la tâche du moteur de synthèse. Toutefois, pour que ce dernier puisse gérer le grand nombre de règles que cela produirait, il nous reste encore à implémenter un algorithme de matching des règles qui soit suffisamment efficace.

Enfin, nous espérons qu'une version complète de *Sentence Garden* verra le jour et que nous pourrons voir ici et là quelques linguistes, la bêche à la main, faire pousser dans leur potager des phrases savoureuses.

Index des notions

A

actant 46, 47
arbre 38
arc 33, 36
article de dictionnaire 52
attribut (d'un objet) 23

C

cible 71
classe (d'objets) 23
concurrence 20
conditions (d'une règle) 40
Customs . . . . . 57, xxv, xxvi

D

DEC 44, 53, 60
diagnostic 24
DiCo . . . . . 44, 53, 57, 60, xi
dictionnaire . . . . . 44, 49, 53, 60, xv

E

élément (d'une représentation) . . . 20
élément actif (d'une règle) 40
élément contextuel (d'une règle) . 41
étiquette (d'un nœud) 33
étiquette sémantique . . 26, 46, 50, 59
exploration (d'une source) 66

F

fonction lexicale 47, 50

G

générateur de messages 15, 26
générateur de textes 14
grammaire . . . . . 39, 41, xiii, xvi
graphe 37

H

hiérarchie communicative 41, 66

I

indice (d'un nœud) 33
instance (d'une classe) 23

J

JEditGraph . . . . . 41, xii

L

LAF 44, 47, 48, 58
lexie 45, 52

M

matching 72
MATE 41, 74
méthode (d'une classe) 23
microsynthèse 18, 64
module (d'un MST) 40, 42
MST 20, 26, 39, 64

N

nanosynthèse 21
nœud 33, 36
nœud d'entrée 66, 67

O

objet (informatique) 23

P

parcours (d'une source) 64
partie du discours 46, 59
partie gauche/droite (d'une règle) . 40
produit 18

R

réalisation 19
régime (d'une lexie) . . 47, 50, 53, 60
règle . . . . . .40, 42, xiii
règle d'équivalence 21
représentation linguistique . . . 33, 36
RSém 34, 38
RSyntP 35, 39

S

Sentence Garden . . . . . 23, 28, 63, xiv
sommet (syntaxique) 35, 67
source 18
SSém 34
SSém-Comm 34
SSém-Rhét 34
SSynt-AnaphP 35
SSynt-CommP 35
SSyntP 35
structure actancielle 46, 50
syntactique 45, 50, 59
synthèse 17, 63
synthèse de la parole 15
synthétiseur de textes 15

T

transition 18, 71
transition élémentaire 20, 40
transitions concurrentes 20, 71

U

unification 73

V

valeur fusionnée (d'une FL) 49
vocable 45, 53

Références

Apresjan, Ju. D., I. M. Boguslavsky, L. L. Iomdin *et al*. 1992. ‘ ETAP-2 : The linguistics of a machine translation system ’. *META*, 37(1) : 97-112.

Bohnet, B. 1999. *Entwurf und Implementierung eines intelligenten Werkzeugs für die Entwicklung von Generierungsgrammatiken* [Architecture et implémentation d’un outil intelligent pour les grammaires de génération], mémoire de maîtrise, Institut für Informatik, Universität Stuttgart.

Bohnet, B., A. Langjahr & L. Wanner. 2000. ‘ A development environment for an MTT-based sentence generator ’. In : *Proceedings of the First International Conference on Natural Language Generation*, Mitzpe Ramon, Israel : 260-263.

Boyer, M. & Lapalme, G. 1985. *Generating Paraphrases from Meaning-Text Semantic Networks*, publication no. 522, Département d’informatique et de recherche opérationnelle, Université de Montréal.

Coch, J. 1996. ‘ Overview of AlethGen ’. In : *Proceedings of the Eighth International Workshop on Natural Language Generation (INLG-1996) (Demonstrations and Posters)* : 25-28.

Delannoy, C. 1998. *Programmer en langage C++*, Paris : Eyrolles.

Goldberg, E., N. Driedger & R. Kittredge. 1994. ‘ Using natural language processing to produce weather forecasts ’. *IEEE Expert*, 9(2) : 45-53.

Iordanskaja, L. 1961. *Dva operatora obrabotki slovosočetanij s "sil´nym upravleniem" (dlja avtomatičeskogo sintaksičeskogo analiza)* [Deux opérateurs pour le traitement des syntagmes avec « régime fort » (dans le cadre de l’analyse syntaxique automatique)], Moscou : Institut de linguistique de l’Académie des sciences de l’URSS.

Iordanskaja, L., R. Kittredge, & A. Polguère. 1988. ‘ Implementing a Meaning-Text model for language generation ’. Supplément à : *Proceedings of the 12th International Conference on Computational Linguistics (COLING-88)*, Budapest : 22-27.

Iordanskaja, L., M. Kim, R. Kittredge, B. Lavoie & A. Polguère. 1992. ‘ Generation of extended bilingual statistical reports ’. In : *Proceedings of the 14th International Conference on Computational Linguistics (COLING-1992)*, vol. 3 : 1019-1023.

Iordanskaja, L. & I. A. Mel’čuk. 2000. ‘ The notion of surface-syntactic relation revisited (valence-controlled surface-syntactic relations in French) ’. In : *Slovo v tekste i v slovare. Sbornik statej k semidesjatiletiju Ju. D. Apresjana* [Le mot

dans le texte et dans le lexique. Recueil d'articles pour le 70e anniversaire de Ju. D. Apresjan], sous la direction de L.L. Iomdin et L.P. Krysin, Moscou : Jazyki russkoj kul'tury [les langues de la culture russe]: 391-433. Une version légèrement différente (plus longue) est également disponible : http://www.fas.umontreal.ca/LING/olst/FrEng/SSyntRels.pdf

Kahane, S. & A. Polguère. 2001. ' Formal foundation of lexical functions '. In : *Workshop proceedings of COLLOCATION : Computational Extraction, Analysis and Exploitation, 39th Annual Meeting and 10th Conference of the European Chapter of the Association for Computational Linguistics*, Toulouse : 8-15. http://www.fas.umontreal.ca/ling/olst/FrEng/SKAPolColloc.pdf

Lavoie, B. & O. Rambow. 1997. ' A fast portable realizer for text generation '. In : *Proceedings of the Fifth Conference on Applied Natural Language Processing (ANLP-1997)* : 265-268.

Mangeot-Lerebours, M. 2000. ' Papillon lexical database project : monolingual dictionaries and interlingual links '. In : *Proceedings of the 7th Workshop on Advanced Information Network and System, Pacific Association for Computational Linguistics 1997 Conference (WAINS'97)*, sous la direction de H. Sakaki, Bangkok : Kasetsart University. Une version électronique est également disponible dans le site http://www.papillon-dictionary.org.

Mel'čuk, I. A. 1987. *Surface syntax of English. A Formal Model within the Meaning-Text Framework*, Amsterdam/Philadelphia : J. Benjamins.

Mel'čuk, I. A. 1993. *Cours de morphologie générale : théorique et descriptive*, vol. 1. Montréal : Presses de l'Université de Montréal / Paris : CNRS.

Mel'čuk, I. A. 1997. *Vers une linguistique Sens-Texte. Leçon inaugurale*, Paris : Collège de France. http://www.fas.umontreal.ca/ling/olst/FrEng/melcukCollde Fr.pdf

Mel'čuk, I. A. 2000. *Cours de morphologie générale : théorique et descriptive*, vol. 5. Montréal : Presses de l'Université de Montréal / Paris : CNRS.

Mel'čuk, I. A. 2001. *Communicative Organization in Natural Language : The Semantic-Communicative Structure of Sentences*, Amsterdam/Philadelphia : J. Benjamins.

Mel'čuk, I. A. À paraître. *Dependency in linguistic description*. http://www.fas.umontreal.ca/LING/olst/FrEng/Dependency.pdf

Mel'čuk, I. A., A. Clas & A. Polguère. 1995. *Introduction à la lexicologie explicative et combinatoire*, Louvain-la-Neuve : Duculot.

Mel'čuk, I. A. *et al.* 1984, 1988, 1992, 1999. *Dictionnaire explicatif et combinatoire du français contemporain : Recherches lexico-sémantiques I, II, III, IV.* Montréal : Presses de l'Université de Montréal.

Milićević, J. 1997. *Étiquettes sémantiques dans un dictionnaire formalisé du type Dictionnaire Explicatif et Combinatoire*, mémoire de maîtrise, Département de linguistique et de traduction, Université de Montréal.

Milićević, J. À paraître. *Modélisation sémantique, lexicale et syntaxique de la paraphrase*, thèse de doctorat, Département de linguistique et de traduction, Université de Montréal.

Nasr, A. 1996. *Un modèle de reformulation de phrases fondé sur la Théorie Sens-Texte. Application aux langues contrôlées*, thèse de doctorat, UFR d'informatique, Université Paris 7.

Polguère, A. 1990. *Structuration et mise en jeu procédurale d'un modèle linguistique déclaratif dans un cadre de génération de texte*, thèse de doctorat, Département de linguistique et de traduction, Université de Montréal. http://www.fas.umontreal.ca/ling/olst/FrEng/APolPhD1990.pdf

Polguère, A. 1997a. ' Meaning-Text semantic networks as a formal language '. In: *Recent Trends in Meaning-Text Theory*, sous la direction de L. Wanner, Amsterdam/Philadelphia: J. Benjamins.

Polguère, A. 1997b. ' Engineering text generation '. *La Tribune des industries de la langue et de l'information électronique*, 23-24, Paris: AUPELF-UREF: 21-29. http://www.fas.umontreal.ca/ling/olst/FrEng/EnginTG.pdf

Polguère, A. 1998. ' La théorie Sens-Texte '. *Dialangue*, 8-9, Université du Québec à Chicoutimi: 9-30. http://www.fas.umontreal.ca/ling/olst/FrEng/PolgIntroTST.pdf

Polguère, A. 2000a. ' Une base de données lexicales du français et ses applications possibles en didactique '. *Revue de Linguistique et de Didactique des Langues (LIDIL)*, 21: 75-97.

Polguère, A. 2000b. ' Towards a theoretically-motivated general public dictionary of semantic derivations and collocations for French '. In: *Proceedings of EURALEX'2000*, Stuttgart: 517-527. http://www.fas.umontreal.ca/ling/olst/FrEng/APolEURALEX.pdf

Polguère, A. 2000c. ' A "Natural" Lexicalization model for language generation '. In: *Proceedings of the Fourth Symposium on Natural Language Processing 2000 (SNLP'2000)*, Chiangmai, 10-12 mai 2000: 37-50. http://www.fas.umontreal.ca/LING/olst/FrEng/SNLP2000.pdf

Reiter, E. & R. Dale. 2000. *Building Natural Language Generation Systems*. Cambridge:Cambridge University Press.

Tomita, M. 1988. ' Graph structured stack and natural language parsing '. In: *26th Meeting of the Association for Computational Linguistics (ACL'88)*, Buffalo.

Wanner, L. 1996. ' Lexical choice in text generation and machine translation '. *Machine Translation Journal*, 19, 1-3.

Zadeh, L.. 1965. ' Fuzzy sets '. *Information and Control*, 8: 338-353.

Žolkovskij, A. & I. A. Mel'čuk. 1967. ' O semantiãeskom sinteze [Sur la synthèse sémantique] '. *Problemy kibernetiki*, 19: 177-238. Une traduction française de ce texte a été publiée dans *T.A. Informations*, 1970 (2).

Annexe 1. Un article du *DiCo*

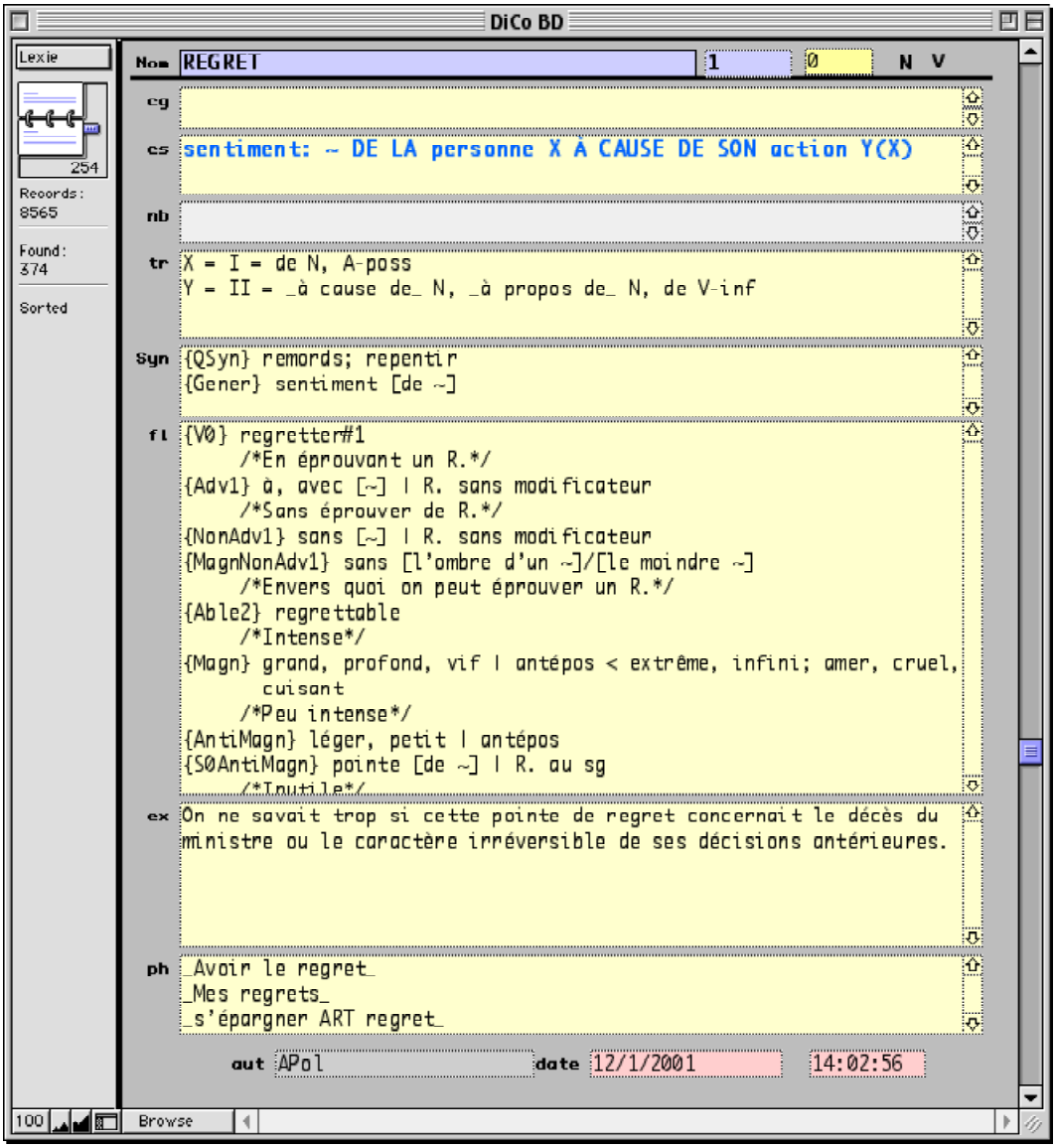


Figure 18. Un article du DiCo *dans* FileMaker Pro*:* REGRET1

Annexe 2. Le logiciel *JEditGraph*

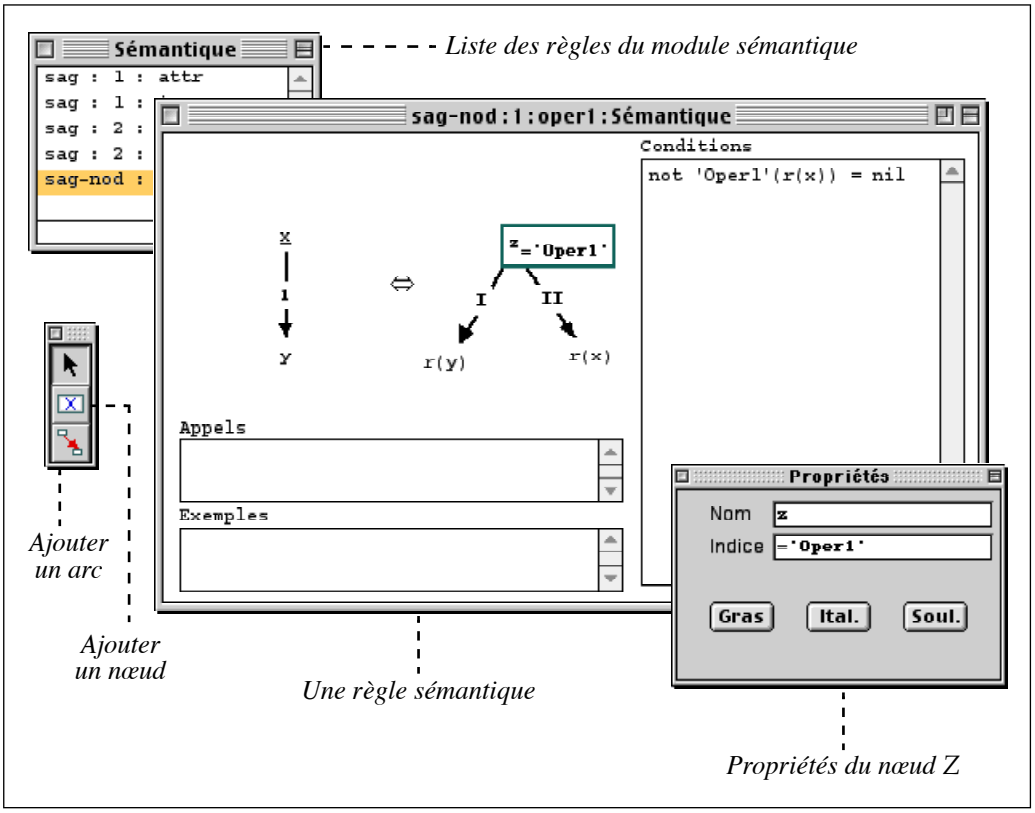
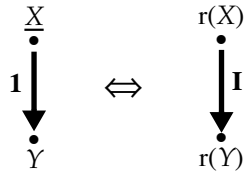


Figure 19. L'environnement JEditGraph

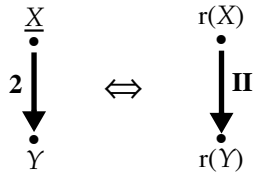
Annexe 3. Quelques règles de grammaire

Règle 3. RelSém 1 ⇔ RelSyntP I



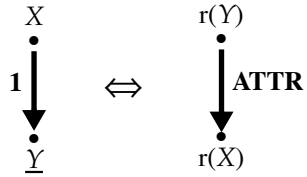
Exemple : ‘Jean’ ***←1—*** ‘aimer’ *—2→* ‘Marie’
⇔
JEAN ***←I—*** AIMER *—II→* MARIE
Jean aime Marie

Règle 4. RelSém 2 ⇔ RelSyntP II



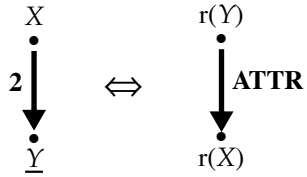
Exemple : ‘Jean’ *←1—* ‘aimer’ ***—2→*** ‘Marie’
⇔
JEAN *←I—* AIMER ***—II→*** MARIE
Jean aime Marie

Règle 5. RelSém 1 ⇔ RelSyntP ATTR



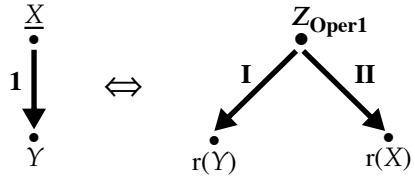
Exemple : ‘Jean’ ***←1—*** ‘aimer’ *—2→* ‘Marie’
⇔
JEAN ***—ATTR→*** AIMER *—II→* MARIE
Jean qui aime Marie

Règle 6. RelSém 2 ⇔ RelSyntP ATTR



Exemple : ‘Jean’ *←1—* ‘aimer’ ***—2→*** ‘Marie’
⇔
MARIE ***—ATTR→*** AIMER *—I→* JEAN
Marie que Jean aime

Règle 7. Introduction d'un verbe support



$Oper_1(r(X)) \neq \emptyset$

Exemple : ‘Jean’ ***←1—*** ‘dormir’
⇔
JEAN ***←I— Oper$_1$ —II→*** DODO
Jean fait dodo

Annexe 4. Une récolte

Voici le contenu de quelques fichiers de sortie construits par le système d'après l'exemple décrit au chapitre II. Dans ce qui suit, tout le texte souligné correspond à un lien hypertexte dans les fichiers originaux. Dans les représentations sémantiques et syntaxiques, les numéros entre parenthèses correspondent aux numéros d'identification internes des objets informatiques. Il permettent d'identifier sans ambiguïté n'importe quel élément dans les représentations et sont utilisés pour y référer.

harvest.html

```
semr5
  |---> dsyntr35
  |---> dsyntr53
  |---> dsyntr48

Variety: Français
  Dictionary
  Grammar
```

dictionary.html

Voici un exemple d'article de dictionnaire qui a été chargé en mémoire. Pour une meilleure lisibilité, nous omettons certaines informations présentes dans le fichier d'origine. De plus, l'information a été légèrement reformattée.

```
============ REGRET#1 ============
• Vocable : REGRET#0
• Government pattern :
     [ [  (X,I,[ de N,A-poss] ),
          (Y,II,[ _à cause de_ N,_à propos de_ N,de V-inf] ) ] ]
• Lexical functions :
  Gener,lexeme:SENTIMENT,de ~,?
  QSyn,lexeme:REPENTIR,?,?
  QSyn,lexeme:REMORDS,?,?
  Magn•IncepFunc1,lexeme:ASSAILLIR,N=X,?
  Oper21,lexeme:PROVOQUER,du/des ~(s) chez N=X,?
  Oper21,lexeme:LAISSER,des ~s à N=X,?
  Oper21,lexeme:CAUSER,ART ~ à/chez N=X,?
  Magn•Oper1,phrase:être rongé,de ~s,?
  Magn•Oper1,lexeme:SE CONSUMER,de ~,?
  Oper1,lexeme:RESSENTIR,ART ~,?
  Oper1,lexeme:ÉPROUVER,?,?
  Oper1,lexeme:AVOIR,?,?
  AntiVer-utilité,lexeme:VAIN,?,antépos
  S0AntiMagn,lexeme:POINTE,de ~,R. au sg
  AntiMagn,lexeme:PETIT,?,antépos
  AntiMagn,lexeme:LÉGER,?,?
  Magn,lexeme:CUISANT,?,?
  Magn,lexeme:CRUEL,?,?
  Magn,lexeme:AMER,?,?
  Magn,lexeme:INFINI,?,?
  Magn,lexeme:EXTRÊME,?,?
  Magn,lexeme:VIF,?,antépos
  Magn,lexeme:PROFOND,?,?
  Magn,lexeme:GRAND,?,?
  Able2,lexeme:REGRETTABLE,?,?
  MagnNonAdv1,lexeme:SANS,l'ombre d'un ~/le moindre ~,?
  NonAdv1,lexeme:SANS,~,R. sans modificateur
  Adv1,lexeme:AVEC,~,R. sans modificateur
  Adv1,lexeme:À,?,?
  V0,lexeme:REGRETTER#1,?,?
• Examples :
  On ne savait trop si cette pointe de regret concernait le décès du
  ministre ou le caractère irréversible de ses décisions antérieures.
```

grammar.html

Toutes les règles de la grammaire sont énumérées dans le fichier `grammar.html` sous forme de tables. Elles ont été légèrement reformattées ici pour une meilleure lisibilité. Nous ne mettons que quelques règles à titre d'exemple.

La règle suivante a été créée automatiquement lors du chargement de l'article de dictionnaire pour REGRET**1**.

lex:abdication#2:ABDICATION#2		
node(x) **label(x,regret#1)**	⇔	**node(r(x))** **label(r(x),REGRET#1)**

Les cinq règles suivantes correspondent aux règles 3 à 7 de l'annexe 3.

sag:1:i		
node(x) node(y) **arrow(y,1,x)**	⇔	node(r(x)) node(r(y)) **arrow(r(x),I,r(y))**

sag:2:ii		
node(x) node(y) **arrow(y,2,x)**	⇔	node(r(x)) node(r(y)) **arrow(r(x),II,r(y))**

sag:1:attr		
node(x) dominant(x) node(y) **arrow(y,1,x)**	⇔	node(r(x)) node(r(y)) **arrow(r(x),ATTR,r(y))**

sag:2:attr		
node(x) dominant(x) node(y) **arrow(y,2,x)**	⇔	node(r(x)) node(r(y)) **arrow(r(x),ATTR,r(y))**

sag-nod:1:oper1		
node(x) dominant(x) node(y) **arrow(x,1,y)**	⇔	**node(z)** **label(z,Oper1)** node(r(y)) node(r(x)) **arrow(z,I,r(y))** **arrow(z,II,r(x))**
Conditions: not Oper1(r(x))=nil		

Viennent ensuite des règles d'équivalence sémantique (*cf.* ch. I§ 4, p. 17). Ces règles sont créées automatiquement lors du chargement des articles de dictionnaire. Par exemple, le chargement de REGRET**1** a entraîné la création des trois règles suivantes. On remarque que ces règles correspondent à certaines fonctions lexicales de la lexie en question (voir l'annexe 1).

equiv:lex:REGRET#1:QSyn:REMORDS		
node(x) **label(x,regret#1)**	⇔	**node(r(x))** **label(r(x),remords)**

equiv:lex:REGRET#1:QSyn:REPENTIR		
node(x) **label(x,regret#1)**	⇔	**node(r(x))** **label(r(x),repentir)**

equiv:lex:REGRET#1:V0:REGRETTER#1		
node(x) **label(x,regret#1)**	⇔	**node(r(x))** **label(r(x),regretter#1)**

semr5.html

```
(10) regret#1    rheme
  |
  |(11)--1--> (7) abeille    theme
  |
  |(12)--2--> (6) abandon#I.3a    rheme
              |
              |(13)--1--> (7) abeille
              |
              |(14)--2--> (8) tigre    rheme

(9) fier#II    theme
  |
  |(15)--1--> (7) abeille
```

Ceci est équivalent à la RSém illustrée à la figure 8, p. 28.

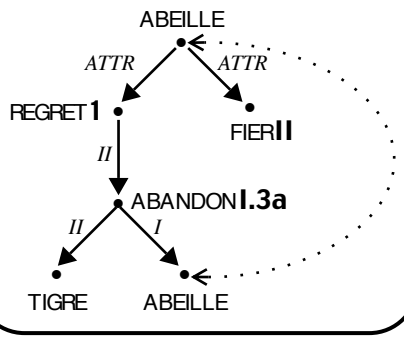
Position in the plant
semr5
|---> dsyntr35
|---> dsyntr53
|---> dsyntr48

Go back to the harvest summary

dsyntr35.html

```
(36) ABEILLE
  |
  |(40)--ATTR--> (39) REGRET#1
  |                |
  |                |(42)--II--> (41) ABANDON#I.3a
  |                |                |
  |                |                |(43)--I--> (46) ABEILLE
  |                |                |
  |                |                |(45)--II--> (44) TIGRE
  |
  |(38)--ATTR--> (37) FIER#II
```

Cet arbre se lit comme suit :



Rules applied

lex:abeille:ABEILLE		
7 label(7)	⇔	36 label(36)

lex:fier#II:FIER#II		
9 label(9)	⇔	37 label(37)

sag:1:attr		
15	⇔	38

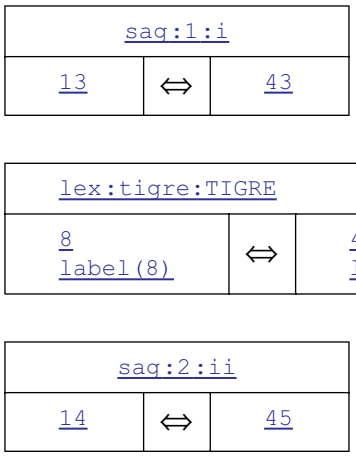
lex:regret#1:REGRET#1		
10 label(10)	⇔	39 label(39)

Cette table indique que c'est la règle `lex:regret#1:REGRET#1` qui a permis d'établir la correspondance entre le nœud `10` ainsi que son étiquette et le nœud `39` et son étiquette.

sag:1:attr		
11	⇔	40

lex:abandon#I.3a:ABANDON#I.3a		
6 label(6)	⇔	41 label(41)

sag:2:ii		
12	⇔	42

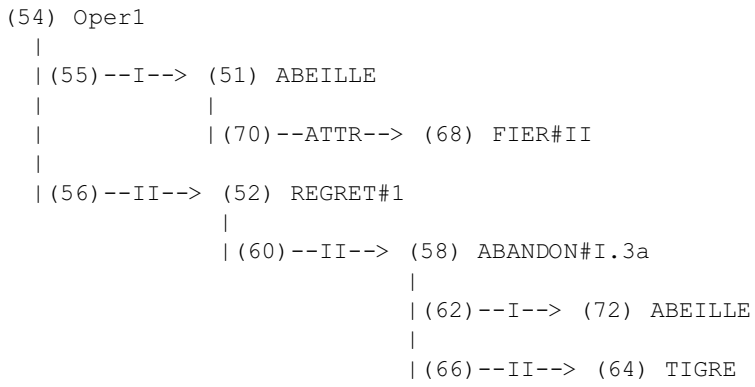


Node correspondences
7 (abeille) <=> 46 (ABEILLE)
7 (abeille) <=> 36 (ABEILLE)
9 (fier#II) <=> 37 (FIER#II)
10 (regret#1) <=> 39 (REGRET#1)
6 (abandon#I.3a) <=> 41 (ABANDON#I.3a)
8 (tigre) <=> 44 (TIGRE)

Position in the plant
semr5
|---> **dsyntr35**
|---> dsyntr53
|---> dsyntr48

Go back to the harvest summary

dsyntr53.html



Rules applied

lex:regret#1:REGRET#1		
10 label(10)	⇔	52 label(52)

lex:abeille:ABEILLE		
7 label(7)	⇔	51 label(51)

saq-nod:1:oper1		
11	⇔	54 label(54) 55 56

lex:abandon#I.3a:ABANDON#I.3a		
6 label(6)	⇔	58 label(58)

saq:2:ii		
12	⇔	60

saq:1:i		
13	⇔	62

lex:tigre:TIGRE		
8 label(8)	⇔	64 label(64)

sag:2:ii		
14	⇔	66

lex:fier#II:FIER#II		
9 label(9)	⇔	68 label(68)

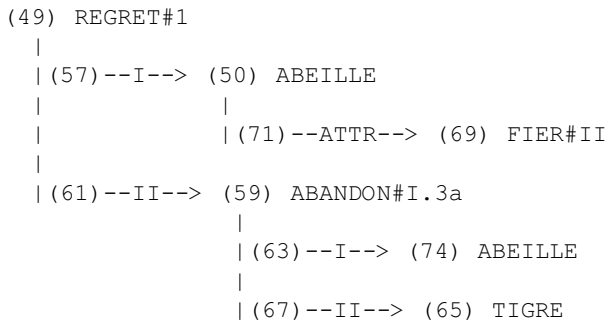
sag:1:attr		
15	⇔	70

Node correspondences
7 (abeille) <=> 72 (ABEILLE)
10 (regret#1) <=> 52 (REGRET#1)
7 (abeille) <=> 51 (ABEILLE)
6 (abandon#I.3a) <=> 58 (ABANDON#I.3a)
8 (tigre) <=> 64 (TIGRE)
9 (fier#II) <=> 68 (FIER#II)

Position in the plant
semr5
|---> dsyntr35
|---> **dsyntr53**
|---> dsyntr48

Go back to the harvest summary

dsyntr48.html



Rules applied

lex:regret#1:REGRET#1		
10 label(10)	⇔	49 label(49)

lex:abeille:ABEILLE		
7 label(7)	⇔	50 label(50)

sag:1:i		
11	⇔	57

lex:abandon#I.3a:ABANDON#I.3a		
6 label(6)	⇔	59 label(59)

sag:2:ii		
12	⇔	61

sag:1:i		
13	⇔	63

lex:tigre:TIGRE		
8 label(8)	⇔	65 label(65)

saq:2:ii		
14	⇔	67

lex:fier#II:FIER#II		
9 label(9)	⇔	69 label(69)

saq:1:attr		
15	⇔	71

Node correspondences
7 (abeille) <=> 74 (ABEILLE)
10 (regret#1) <=> 49 (REGRET#1)
7 (abeille) <=> 50 (ABEILLE)
6 (abandon#I.3a) <=> 59 (ABANDON#I.3a)
8 (tigre) <=> 65 (TIGRE)
9 (fier#II) <=> 69 (FIER#II)

Position in the plant
semr5
|---> dsyntr35
|---> dsyntr53
|---> **dsyntr48**

Go back to the harvest summary

Annexe 5. L'importation du dictionnaire

```
Customs
=======

Government patterns have to be imported before the dictionary.

To import a government pattern file : "approveGovPat( Path )."
To import a dictionary file         : "approveDictionary( Path )."

yes
| ?- approveDictionary( '/HD/users/francois/Sentence Garden/
Varieties/Français/sources/dico court' ) .
Indexing the file...
Checking À L'ARRIÈRE-PLAN
Checking ABAISSEMENT (3 lexical units)
Checking ABANDON (8 lexical units)
Checking ABAT-JOUR
Checking ABATTEMENT(1)
Checking ABATTEMENT(2)
Checking ABCÈS (2 lexical units)
Checking ABDICATION (2 lexical units)
Checking ABDIQUER (2 lexical units)
Checking ABEILLE
[…]
Checking TIR (4 lexical units)
Checking TIRAGE(1) (3 lexical units)
Checking TIRAGE(2)
Checking TIRAGE(3)
Checking TISANE
Checking TÊTE-À-QUEUE
Checking VICTOIRE (2 lexical units)

Dictionary file created: '/HD/users/francois/Sentence Garden/
Varieties/Français/Dictionary'
See the report file: '/HD/users/francois/Sentence Garden/Components/
Customs/Reports/Dict import 2002.08.18_14.24.53'
```

Annexe 6. Un rapport de *Customs*

Le texte a été légèrement reformatté pour une meilleure lisibilité.

```
BARBE
=======================================================
(ERROR) Pointers are not accepted in this version:
     {couleurs de la B.} **HERE**->|| poil#a
(WARNING) Smart quotes should be used:
     {{B. en tant qu**HERE**->'élément esthétique}}
This entry was accepted with modifications


CRIMINEL (2 lexical units)
=======================================================
CRIMINEL#0
This entry was accepted

CRIMINEL#I
(ERROR) This ƒ is not defined:
    {Essayer de Caus**HERE**->ƒ} interroger [ART ~]
This entry was accepted with modifications

CRIMINEL#II
This entry was accepted

The vocable was accepted with modifications


FARINE
=======================================================
(WARNING) This ƒ definition should be in the LAF part:
    ...Labreal◊ **HERE**->[ =ƒ]} rouler [N dans la ~]
This entry was accepted
```